

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
GRADUATION PROJECT (CO4337)

INTELLIGENT CODE ASSISTANCE FOR WEB DEVELOPMENT: RAG-ENHANCED LLM APPROACH

Major: Computer Science

Supervisors: Nguyen An Khuong, PhD
Pham Nhut Huy, Engineer - Zalo AI

Secretary:

Students: Nguyen Trang Sy Lam – Student ID: 2152715
Bui Ho Hai Dang – Student ID: 2153289
Le Hoang Phuc – Student ID: 2152239

Ho Chi Minh City, May 2025

PhD Nguyen An Khuong (Supervisor)

Date: _____

Declaration

We hereby declare that this research is entirely our own work, conducted under the supervision and guidance of Dr. Nguyen An Khuong and Mr. Pham Nhut Huy. The results presented in this research are legitimate and have not been published in any form prior to this. All materials used in this research have been collected by us from various sources and are properly cited in the bibliography section. Additionally, all figures included in this report have been created by us.

This research also references results from other authors, which have already been published by the respective authors.

In the event of any plagiarism, we take full responsibility for our actions. Ho Chi Minh City University of Technology - Vietnam National University is not liable for any copyright infringements arising from this research.

Ho Chi Minh City, May 2025

Authors

Acknowledgements

Firstly, we would like to sincerely express our gratitude to our supervisors, Dr. Nguyen An Khuong, and Mr. Pham Nhut Huy. To Dr. Nguyen An Khuong, thank you for dedicating your precious time to us. The weekly meetings helped our team stay on track, and your timely feedback, advice, and guidance greatly contributed to the success of our research. To Mr. Pham Nhut Huy, your expertise in the field, along with your thesis writing and presentation skills, was invaluable to our progress.

Secondly, we wish to extend our heartfelt thanks to our families for their unwavering support, both emotionally and financially, throughout this research process. Our accomplishments would not have been possible without their encouragement and assistance.

Lastly, this is from Nguyen Trang Sy Lam. I would like to personally thank Dr. Nguyen An Khuong for guiding me through the most challenging days of my student life. When I was in a dark place, your support and mentorship pulled me through and helped me complete this research. Words cannot fully convey my gratitude to you.

Abstract

In this project, we present the design of a Retrieval-Augmented Generation (RAG) workflow integrated into a Visual Studio Code (VS Code) extension, serving as a local intelligent assistant for web developers working with the Next.js framework. The system uses Milvus [48] as the vector database to efficiently store and retrieve semantically indexed information from the official Next.js documentation, prepared during the data processing phase.

For response generation, the architecture is model-agnostic and designed to work with locally hosted Large Language Models (LLMs) via Ollama¹, a lightweight framework that allows users to run various LLMs depending on their hardware capabilities. This flexibility empowers developers to choose the model that best suits their system—whether it’s a smaller model for lightweight environments or a more powerful one for advanced tasks.

By combining retrieval from up-to-date documentation with locally generated, context-aware responses, the proposed RAG-enhanced assistant boosts productivity and simplifies development workflows directly within the VS Code environment.

¹Ollama - <https://ollama.com/>

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
1 Introduction	1
1.1 Motivations	1
1.2 Problem statement	2
1.3 Significant of the problem	3
1.4 Objectives	3
1.5 Scope	3
1.6 Challenges	3
1.7 Structure of the report	4
2 Preliminaries	5
2.1 Retriever	5
2.1.1 Preprocessing	6
2.1.2 Embedding	6
2.1.3 Hierarchical navigable small world (HNSW) indexing	7
2.1.4 Pre-retrieval	10
2.1.5 Retrieval	11
2.1.6 Post-retrieval	12
2.2 Generator	13
2.2.1 Transformer architecture	13
2.2.2 Prompt engineering	14
2.3 Augmentation methods	15
2.3.1 Augmentation stage: Inference	15
2.3.2 Augmentation source: Structured data	16
2.3.3 Augmentation process	17
3 Related works and tools	20
3.1 RAG and LLMs in coding tasks	20
3.1.1 Retrieval strategies	20
3.1.2 Generative enhancements	22
3.1.3 Specialized Applications	24
3.2 Developer assistance tools	24
3.2.1 Functionality	24
3.2.2 Retrieval mechanisms	25
3.2.3 Code generation capabilities	26
3.2.4 Integration in development processes	26

4	Approach	28
4.1	Fine-tuning vs. RAG	28
4.1.1	Fine-tuning	28
4.1.2	Comparison between fine-tuning and RAG	29
4.1.3	Rationale for choosing RAG	29
4.2	Why BGE-M3?	30
4.3	Why Ollama's LLMs	30
4.4	Why Docker?	30
4.5	Why VS Code Extension?	31
4.6	Main architecture	31
4.6.1	Local vector storage	33
4.6.2	VS Code integration	35
5	Implementation	37
5.1	Database	37
5.1.1	Data Preparation	37
5.1.2	Data Preprocessing	38
5.1.3	Database design	39
5.1.4	Database setup	43
5.1.5	Database API	43
5.2	RAGGIN core services	45
5.2.1	Retriever	45
5.2.2	Generator	47
5.3	RAGGIN VSCode Extension Development	49
5.3.1	Installing in VSCode (Private installation)	49
5.3.2	Icons & labels	49
5.3.3	Primary function	53
5.3.4	Performance	54
5.3.5	Known Limitations and Future Roadmap	54
6	Evaluation	55
6.1	Evaluation metrics	55
6.1.1	Retriever Metrics	55
6.1.2	Generator Metrics	56
6.2	Evaluation methodology	56
6.3	Evaluation summary	56
6.3.1	Retriever Performance	56
6.3.2	Generator Performance	57
6.4	Testing	57
6.4.1	Unit testing	57
6.4.2	Integration testing	57
7	Conclusion	58
7.1	Achieved Results	58
7.2	Limitations	58
7.3	Future development directions	59
	Bibliography	60
	Appendices	64
	A Interface of the RAGGIN extension	65
	B Evaluation and test results	69

1.1 Motivations

The process of web development has been significantly transformed by the introduction and adoption of frameworks. Frameworks have become the industry standard, providing developers with structured, reusable tools that simplify the creation of web applications. This evolution has also been greatly influenced by the rise of LLMs like OpenAI's Codex [9], Google's Gemini [46], and GitHub Copilot¹, which have made coding more accessible by offering automated code generation, real-time suggestions, and debugging assistance. These tools have made entry into web development easier than ever, lowering barriers for beginners and speeding up development processes.

However, despite these advancements, LLMs still face certain limitations, such as hallucinations, difficulties in referencing external sources, and performance issues when handling specific tasks. A promising solution to mitigate these limitations is using RAG. RAG enhances the generative capabilities of LLMs by retrieving relevant external information before generating content, improving both the accuracy and relevance of the output. This technique is particularly beneficial in domains like web development, where staying up-to-date with framework versions and ensuring compatibility across services is critical. Incorporating RAG helps bridge the gap between rapidly evolving frameworks and the knowledge developers need to make informed decisions about the tools and technologies they use.

¹Github Copilot - <https://copilot.github.com/>

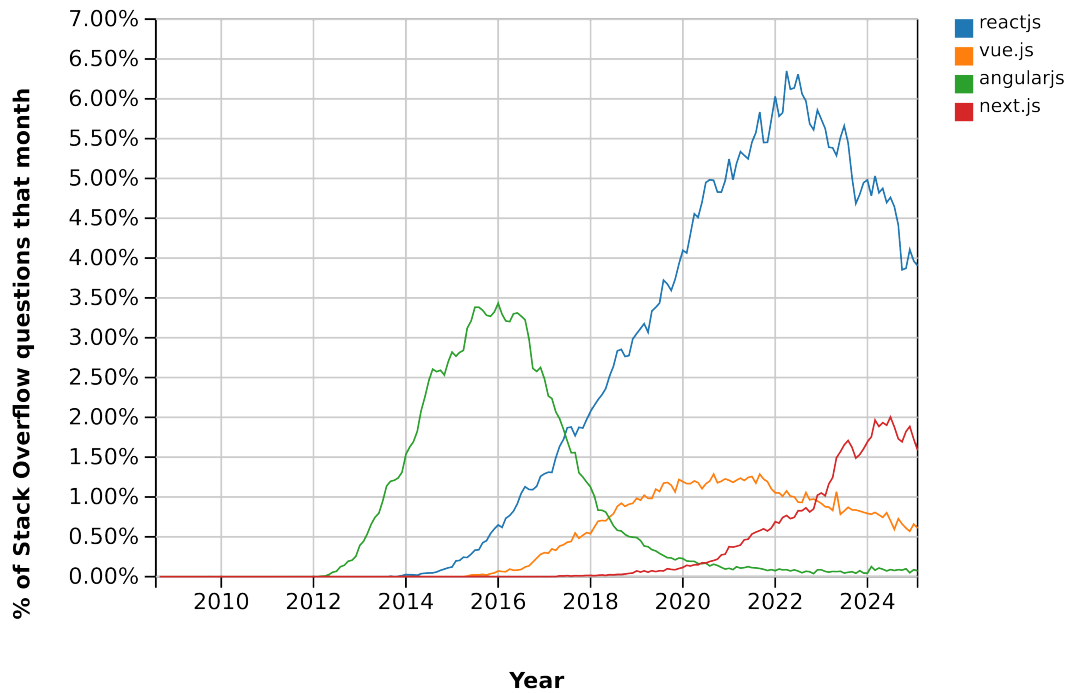


Figure 1.1: The popularity of frameworks based on StackOverflow questions 2008-2025

To illustrate, consider the Next.js framework, which exemplifies the rapid evolution of modern web technologies. Originally built on top of React.js, Next.js has transformed from a simple server-rendering tool into a comprehensive full-stack framework. What makes Next.js particularly notable is the speed at which it introduces significant updates. In just under three years, it moved from version 12 to version 15, each introducing major shifts in architecture and development paradigms.

For instance, Next.js 13 (October 2022) introduced the App Router, React Server Components, and a new file-based routing system—radically altering how pages and layouts are structured. Less than a year later, Next.js 14 (October 2023) refined these features while improving performance with deeper Turbopack integration. And most recently, Next.js 15 (March 2025) pushed the boundaries even further by stabilizing Partial Prerendering, extending Server Actions, and fully embracing React Server Components².

These frequent and impactful changes challenge developers to constantly relearn patterns, adapt to breaking changes, and stay updated with both Next.js and its dependency. React.js, which itself released React 19 with major improvements to transitions, refs, and concurrency³.

1.2 Problem statement

Modern web development frameworks, such as Next.js, offer powerful features and flexibility, making them a popular choice for building high-performance applications. However, as these frameworks continue to evolve, their complexity has grown significantly. Developers are often required to refer to extensive documentation, which can be fragmented and difficult to navigate, to resolve coding challenges, learn new functionalities, or debug issues. This reliance on documentation creates several inefficiencies in the development workflow, including:

- **Time-Consuming Information Retrieval:** Developers frequently need to search through lengthy and intricate documentation to find specific information. Traditional search methods, such as keyword-based tools, often fail to capture the intent behind the query or retrieve relevant context, leading to prolonged and unproductive searches.
- **Fragmented Context:** Even when relevant information is found, it often lacks the necessary context to answer the developer's query fully. This forces developers to cross-reference multiple sections of

²Next.js 15 Release Notes - <https://nextjs.org/blog/next-15>

³React 19 Upgrade Guide - <https://react.dev/blog/2024/04/25/react-19-upgrade-guide>

the documentation, further delaying progress.

- **Disruption of Workflow:** Switching between the code editor and external documentation interrupts the development process, causing a loss of focus and reducing productivity.
- **Inconsistent Results:** Existing tools, such as search engines or static code assistants, often provide inconsistent results that fail to address the specific needs of a query or task, leaving developers to piece together partial solutions on their own.

1.3 Significant of the problem

Solving this problem is of critical importance for several reasons:

- **Enhanced Developer Productivity:** By significantly reducing the time spent searching for relevant information, the proposed system allows developers to focus on coding tasks, accelerating development timelines and increasing overall productivity.
- **Improved Workflow Efficiency:** Delivering context-aware assistance directly within the coding environment (e.g., Visual Studio Code) eliminates the need to switch between multiple tools and resources, streamlining the development process and reducing cognitive load.
- **Support for Complex Frameworks:** As frameworks like Next.js continue to evolve, the volume and complexity of their documentation will only increase. An intelligent system capable of efficiently managing and retrieving this knowledge ensures that developers can stay up-to-date and adapt to new features quickly.

By solving this problem, we aim to not only improve the efficiency of individual developers but also set a foundation for the future of Artificial Intelligent (AI), AI-assisted development environments. This research will demonstrate how advanced AI techniques like RAG can transform the way developers interact with documentation, enhancing both their productivity and their overall experience.

1.4 Objectives

The objective of this research is to develop a tool, specifically a VS Code extension, that facilitates the web development process by addressing common challenges associated with modern frameworks. Utilize RAG to assist developers with question answering, code generation, and code review. RAG will retrieve relevant information from documentation or community forums to enhance the accuracy and relevance of the tool's suggestions.

1.5 Scope

The scope is intentionally confined to Next.js as the primary framework for web application development, enabling a focused exploration of its interaction with RAG methodologies. Additionally, the implementation targets applications written in JavaScript, TypeScript, HTML, and CSS, ensuring compatibility with commonly used web development programming languages.

1.6 Challenges

Developing this tool involves several challenges:

- **Efficient context management:** Since the goal is to make the tool lightweight, storing and managing the context of a user's codebase in a way that allows for fast retrieval without overwhelming system resources is a major challenge.
- **Data curation complexity:** The data curation process requires extensive knowledge about the framework.
- **Handling framework updates:** Keeping the tool updated with the frequent changes in frameworks like Next.js and their associated packages is challenging. Continuous updates are needed to ensure compatibility and provide accurate suggestions.

- User experience: Balancing the amount of information presented to the user without overwhelming them is essential. The tool should provide meaningful insights and recommendations without causing information overload.

1.7 Structure of the report

Chapter 1. Introduction. This chapter introduces the motivations, objectives, and scope of the research. It highlights the challenges of web development and the role of RAG in enhancing efficiency and accuracy. The chapter outlines the structure of the report, providing a roadmap for the research presented.

Chapter 2. Preliminaries. This chapter provides foundational knowledge on RAG, emphasizing its retriever and generator components. It discusses indexing, vector embeddings, and augmentation methods, offering a clear understanding of the technologies used.

Chapter 3. Related works and tools. This chapter explores prior research on RAG and its application in web development. It reviews developer tools such as VS Code extensions and discusses strategies for retrieval and augmentation. Comparative analyses of existing frameworks and methodologies are also presented.

Chapter 4. Approach. This chapter presents the reasoning behind adopting RAG over fine-tuning, and explains the selection of key components such as the BGE-M3 embedding model and Ollama’s local LLMs. It also discusses design choices including containerization with Docker and the use of a VS Code extension. Finally, it outlines the system’s core architecture, focusing on local vector storage and editor integration.

Chapter 5. Implementation. This chapter details the development process of the RAGGIN system, covering data preparation, preprocessing, database design and setup, and the implementation of a RESTful API. It also introduces the core retrieval and generation services, and concludes with the integration of these components into a VS Code extension.

Chapter 6. Evaluation. This chapter presents the evaluation of the system through quantitative metrics for both retrieval and generation components. It describes the evaluation methodology, summarizes results, and includes both unit and integration testing to validate system behavior. The analysis highlights key strengths as well as performance limitations under different usage scenarios.

Chapter 7. Conclusion. The final chapter summarizes the key results achieved, acknowledges the current system limitations, and outlines clear directions for future development.

The RAG process combines the capabilities of retrieval systems and generative AI to deliver precise and context-aware responses. It begins with query vectorization, where the user's input is transformed into a mathematical vector using models like Bidirectional Encoder Representations from Transformers (BERT) [11] or sentence transformers. This vector is then used to search a pre-indexed vector database for semantically similar entries, retrieving the most relevant data. Once retrieved, the data undergoes preprocessing to ensure compatibility with the language model, such as summarizing or structuring it. The refined data is then integrated into the input context of a LLM, enabling it to generate responses that blend the retrieved information with its broader pre-trained knowledge. This structured pipeline ensures that the generated outputs are accurate, specific, and contextually grounded.

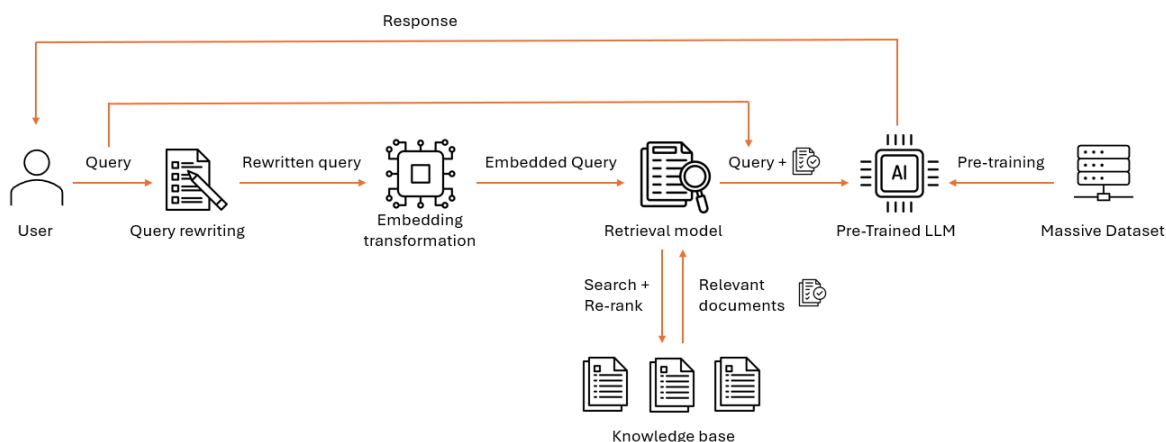


Figure 2.1: RAG Process

The RAG process is composed of two main components: the retriever and the generator. The retriever is responsible for locating and extracting relevant information from an external knowledge source, employing techniques such as keyword-based search, semantic similarity search, or neural network-based retrieval to ensure precise results. The generator then utilizes the retrieved context to produce coherent and accurate responses, ensuring the output aligns with the user's query while maintaining relevance and context. Together, these components form a robust framework for leveraging external knowledge in generative tasks.

2.1 Retriever

The retriever stage involves finding and gathering relevant text passages from an external knowledge source. This can be done through various methods such as keyword-based search, semantic similarity search, or neural network-based retrieval techniques. One major application of embedding models is

neural retrieval. By measuring the semantic relationship with the text embeddings, allowing for the comparison of text based on content rather than just keyword matching, then the relevant answers to the input query can be retrieved based on the embedding similarity.

2.1.1 Preprocessing

Preprocessing starts with the cleaning and extraction of raw data in diverse formats like PDF, HTML, Word, and Markdown, which is then converted into a uniform plain text format. To accommodate the context limitations of language models, text is segmented into smaller, digestible chunks. Chunks are then encoded into vector representations using an embedding model and stored in a vector database. This step is crucial for enabling efficient similarity searches in the subsequent retrieval phase [14].

There are ways to optimize the indexing process to enhance the performance of the upcoming retrieval steps:

1. **Chunking strategy:** Documents can be splitted into chunks using various methods like fixed-size chunking (for structured documents and surveys), recursive chunking (for documents with large amounts of data), sentence-aware chunking (for analysis of textbooks) and semantic chunking (for information retrieval systems and legal document analysis) [23].
2. **Metadata attachments:** Each chunk can include extra information (page number, file name, author, category, or timestamp) as metadata. Afterward, the retrieval process can be refined by filtering results based on metadata, which helps narrow down the search. Assigning different weights to document timestamps during retrieval can achieve time-aware RAG, ensuring the freshness of knowledge and avoiding outdated information [14].

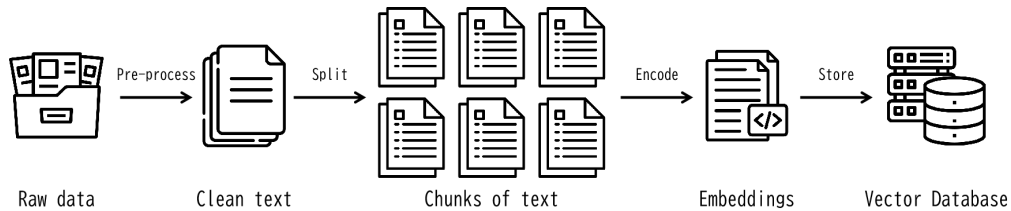


Figure 2.2: Preprocessing process for retriever

2.1.2 Embedding

Embeddings are a key component of RAG systems, which are used to effectively retrieve pertinent information from big datasets. By converting textual data into numerical representations, these embeddings are vector-based representations, typically generated by neural networks, that transform input text into a structured vector space. In this space, proximity reflects the semantic relationships of the input.

In essence, an embedding is a vector, a high-dimensional representation of a text in which every dimension represents a distinct facet of meaning. Consider a two-dimensional Cartesian coordinate system from the algebra class, but with a lot more dimensions (usually 768, 1024, or 1536) to better visualize this. Models such as BERT [11] and Sentence-BERT [38] employ this high-dimensional vector space to represent the semantic features of text. By calculating the distance between these vectors, embeddings enable the system to extract pertinent information from large datasets and choose the most contextually relevant text passages for response generation. This feature is essential for enhancing large language models' (LLMs') outputs' contextual relevance and accuracy.

Many embedding techniques have been used over the years:

1. **Word2Vec** [30]: A well-liked technique that uses the Continuous Bag of Words (CBOW) or Skip-gram models to learn word embeddings depending on local context. Word2Vec is effective at many jobs, but it has a major drawback: it has trouble handling polysemy, which is the situation where words have several meanings depending on the context. This is because Word2Vec creates a single representation for every word, independent of how it is used.
2. **BERT** [11]: A transformer-based model that generates contextual embeddings, considering both the preceding and following words in a sentence. This allows BERT to produce more accurate representations of words based on their surrounding context. It has been widely adopted for many NLP tasks such as question answering, sentiment analysis, and named entity recognition.

3. **Sentence-BERT (SBERT)** [38]: An extension of BERT, fine-tuned specifically for generating sentence embeddings. Unlike traditional BERT, which produces word-level embeddings, SBERT produces fixed-size embeddings for entire sentences. This allows for efficient comparison of sentence pairs, making SBERT highly effective for tasks like semantic textual similarity, paraphrase identification, and information retrieval.
4. **FAISS** [12] and **dense retrieval techniques**: In RAG systems, embeddings are often used with specialized retrieval tools like Facebook AI Similarity Search (FAISS) to perform fast, approximate nearest neighbor search. This helps locate the most relevant pieces of information from a large corpus, making the retrieval process both scalable and efficient.

Moreover, our embedding can be trained in different ways:

1. **Supervised learning**: In supervised learning, embeddings are trained with labeled data to predict specific outcomes, such as class labels or relationships. Examples include training embeddings for tasks like sentiment analysis or named entity recognition (NER).
2. **Unsupervised learning**: Unsupervised learning allows embeddings to be trained without labels, often by predicting words or context from surrounding text. Models like Word2Vec and global vectors for word representation (GloVe) [33] use this method to learn semantic relationships between words from large corpora.

Fine-tune embedding models: Reinforcement learning [24]: In reinforcement learning, embeddings are dynamically adjusted based on feedback from the environment to improve the system’s performance over time. For instance, in a RAG system designed for domain-specific chatbots, reinforcement learning is employed to optimize the retrieval of relevant context. Using reinforcement learning with human feedback (RLHF) [24], models like GPT-4 are fine-tuned to respond accurately to domain-specific queries. A key example is optimizing FAQ retrieval for customer service chatbots. In such systems, reinforcement learning is used to decide whether to fetch additional context for each user query or rely on previously retrieved information. The system evaluates the quality of the generated responses (e.g., by GPT-4) and uses this evaluation as a reward signal. For example, if the system correctly answers a follow-up query without fetching redundant FAQ context, it receives a positive reward, thereby learning to minimize unnecessary retrievals. This approach not only improves efficiency by saving computational resources (such as API token usage) but also enhances accuracy by reducing noise in the generated outputs.

2.1.3 Hierarchical navigable small world (HNSW) indexing

1. Vector database and HNSW:

In RAG systems, vector database plays a pivotal role in efficiently retrieving relevant information to augment the capabilities of generative language models. These databases are designed to store and query high-dimensional embeddings, which are numerical representations of text, code, images, or other data. Embeddings capture semantic relationships between data points, enabling similarity searches based on meaning rather than exact matches. This makes vector databases a cornerstone of modern RAG pipelines, ensuring that retrieved context aligns with the query’s intent. To achieve efficient and scalable retrieval from large embedding datasets, RAG systems often employ approximate nearest neighbor (ANN) algorithms, with HNSW graphs [29] being one of the most widely used techniques.

2. HNSW algorithm:

The HNSW algorithm is built on the concept of organizing links based on their length scales across multiple layers, enabling efficient search within a hierarchical, multilayer graph structure.

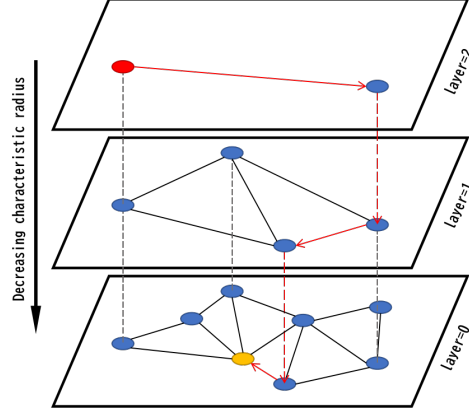


Figure 2.3: HNSW search algorithm visualization. The search starts from entry point from top layer (shown red), red arrows show direction of greedy algorithm to query point (shown yellow) [29]

The HNSW algorithm begins with a network construction phase (Algorithm 1), where the graph structure is incrementally built through the consecutive insertion of stored elements. For each new element, a maximum layer l is randomly determined using an exponentially decaying probability distribution. This hierarchical organization ensures that higher layers contain progressively fewer nodes, optimizing the search process by focusing on coarse-to-fine navigation.

The insertion process begins at the top layer of the hierarchical graph, where the algorithm performs a greedy traversal to locate the ef closest neighbors to the newly inserted element q within that layer. Once these nearest neighbors are identified, they serve as entry points for the next layer. This process is repeated layer by layer, moving downward through the graph, ensuring that the inserted element is effectively connected to its most relevant neighbors at each level.

Algorithm 1 INSERT($hsnw, q, M, M_{\max}, efConstruction, mL$)

Input: multilayer graph $hsnw$, new element q , number of established connections M , maximum number of connections per element per layer $M_{\max}$, size of the dynamic candidate list $efConstruction$, normalization factor for level generation mL

Output: update $hsnw$ inserting element q

```

1:  $W \leftarrow \emptyset$  ▷ list for the currently found nearest elements
2:  $ep \leftarrow$  get enter point for  $hsnw$ 
3:  $L \leftarrow$  level of  $ep$  ▷ top layer for  $hsnw$ 
4:  $l \leftarrow -\lfloor \ln(\text{unif}(0, 1)) \cdot mL \rfloor$  ▷ new element's level
5:  $l \leftarrow \min(l, L + 1)$ 
6: for  $lc \leftarrow L$  to  $l + 1$  do
7:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef = 1, lc)$ 
8:    $ep \leftarrow$  get the nearest element from  $W$  to  $q$ 
9: end for
10: for  $lc \leftarrow \min(L, l)$  to 0 do
11:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, efConstruction, lc)$ 
12:    $neighbors \leftarrow \text{SELECT-NEIGHBORS}(q, W, M, lc)$  ▷ Algorithm 3 or 4
13:   add bidirectional connections from  $neighbors$  to  $q$  at layer  $lc$ 
14:   for each  $e \in neighbors$  do
15:      $eComm \leftarrow neighborhood(e)$  at layer  $lc$ 
16:     if  $|eComm| > M_{\max}$  then ▷ shrink connections if needed
17:       if  $lc = 0$  then ▷ if  $lc = 0$  then  $M_{\max} = M_{\max0}$ 
18:          $M_{\max} \leftarrow M_{\max0}$ 
19:       end if
20:        $eComm \leftarrow \text{SELECT-NEIGHBORS}(e, eComm, M_{\max}, lc)$  ▷ Algorithm 3 or 4
21:     end if
22:     set  $neighborhood(e)$  at layer  $lc$  to  $eComm$ 
23:   end for
24: end for
25:  $ep \leftarrow W$ 
26: set enter point for  $hsnw$  to  $q$ 

```

Closest neighbors at each layer are identified using a variant of the greedy search algorithm (as outlined in Algorithm 2). To approximate the ef nearest neighbors at a given layer l_c , the algorithm maintains a dynamic list W , which stores the ef closest found elements. This list is initially populated with the entry points and is iteratively updated during the search process.

Algorithm 2 SEARCH-LAYER(q, ep, ef, lc)

Input: Query element q , enter points ep , number of nearest to q elements to return ef , layer number lc
Output: ef closest neighbors to q

```

1:  $v \leftarrow ep$  ▷ Set of visited elements
2:  $C \leftarrow ep$  ▷ Set of candidates
3:  $W \leftarrow ep$  ▷ Dynamic list of found nearest neighbors
4: while  $|C| > 0$  do
5:    $c \leftarrow$  Extract nearest element from  $C$  to  $q$ 
6:    $f \leftarrow$  Get furthest element from  $W$  to  $q$ 
7:   if  $\text{distance}(c, q) > \text{distance}(f, q)$  then
8:     break ▷ All elements in  $W$  are evaluated
9:   end if
10:  for each  $e \in \text{neighbourhood}(c)$  at layer  $lc$  do ▷ Update  $C$  and  $W$ 
11:    if  $e \notin v$  then
12:       $v \leftarrow v \cup e$ 
13:       $f \leftarrow$  Get furthest element from  $W$  to  $q$ 
14:      if  $\text{distance}(e, q) < \text{distance}(f, q)$  or  $|W| < ef$  then
15:         $C \leftarrow C \cup e$ 
16:         $W \leftarrow W \cup e$ 
17:        if  $|W| > ef$  then
18:          Remove furthest element from  $W$  to  $q$ 
19:        end if
20:      end if
21:    end if
22:  end for
23: end while
24: return  $W$ 

```

At each step, the algorithm evaluates the neighborhood of the closest, yet-to-be-evaluated element in W . The search continues until the neighborhoods of all elements in W have been evaluated. Unlike traditional approaches that limit the number of distance calculations, the HNSW stop condition offers a key advantage: it discards candidates for evaluation that are farther from the query than the furthest element already in W . This approach prevents unnecessary evaluations and avoids bloating the search structures, ensuring a more efficient and streamlined search process.

In the HNSW algorithm, two methods are employed to select M neighbors from a pool of candidates during the graph construction phase: Simple (Algorithm 3) and Heuristic (Algorithm 4).

Algorithm 3 SELECT-NEIGHBORS-SIMPLE(q, C, M)

Input: Base element q , candidate elements C , number of neighbors to return M

Output: M nearest elements to q

1: **return** M nearest elements from C to q

Algorithm 4 SELECT-NEIGHBORS-HEURISTIC($q, C, M, lc, \text{extendCandidates}, \text{keepPrunedConnections}$)

Input: Base element q , candidate elements C , number of neighbors to return M , layer number lc , flag indicating whether or not to extend candidate list **extendCandidates**, flag indicating whether or not to add discarded elements **keepPrunedConnections**

Output: M elements selected by the heuristic

```

1:  $R \leftarrow \emptyset$ 
2:  $W \leftarrow C$  ▷ Working queue for the candidates
3: if extendCandidates then ▷ Extend candidates by their neighbors
4:   for each  $e \in C$  do
5:     for each  $e_{\text{adj}} \in \text{neighbourhood}(e)$  at layer  $lc$  do
6:       if  $e_{\text{adj}} \notin W$  then
7:          $W \leftarrow W \cup e_{\text{adj}}$ 
8:       end if
9:     end for
10:  end for
11: end if
12:  $W_d \leftarrow \emptyset$  ▷ Queue for the discarded candidates
13: while  $|W| > 0$  and  $|R| < M$  do
14:    $e \leftarrow$  Extract nearest element from  $W$  to  $q$ 
15:   if  $e$  is closer to  $q$  compared to any element from  $R$  then
16:      $R \leftarrow R \cup e$ 
17:   else
18:      $W_d \leftarrow W_d \cup e$ 
19:   end if
20: end while
21: if keepPrunedConnections then ▷ Add some of the discarded connections from  $W_d$ 
22:   while  $|W_d| > 0$  and  $|R| < M$  do
23:      $R \leftarrow R \cup$  Extract nearest element from  $W_d$  to  $q$ 
24:   end while
25: end if
26: return  $R$ 

```

The K-approximate nearest neighbor search (K-ANNS) algorithm, as implemented in the HNSW framework, is detailed in Algorithm 5. It is conceptually similar to the insertion process of an item with a designated top layer $l = 0$. However, instead of establishing new connections, the goal of the K-ANNS algorithm is to identify and return the closest neighbors in the graph's ground layer as the final search results.

Algorithm 5 K-ANN-SEARCH(hsnw, q, K, ef)

Input: Multilayer graph **hsnw**, query element q , number of nearest neighbors to return K , size of the dynamic candidate list ef

Output: K nearest elements to q

```

1:  $W \leftarrow \emptyset$  ▷ Set for the current nearest elements
2:  $ep \leftarrow$  Get enter point for hsnw
3:  $L \leftarrow$  Level of  $ep$  ▷ Top layer for hsnw
4: for  $lc \leftarrow L \dots 1$  do
5:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef = 1, lc)$ 
6:    $ep \leftarrow$  Get nearest element from  $W$  to  $q$ 
7: end for
8:  $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef, lc = 0)$ 
9: return  $K$  nearest elements from  $W$  to  $q$ 

```

2.1.4 Pre-retrieval

Query expansion: In the context of RAG, a single query can be transformed and expanded into multiple variations that are semantically enriched, capturing different nuances and interpretations of the

original input. By diversifying the queries, the system increases the likelihood of retrieving contextually specific and relevant information from the knowledge base. This enriched retrieval process ensures that the generated responses are not only more precise but also better aligned with the user’s intent, ultimately enhancing the quality and relevance of the output provided by the model [14].

Query rewriting: The original queries are not always optimal for LLM retrieval, especially in real-world scenarios. Therefore, we can prompt LLM to rewrite the queries [14]. Query rewriting in RAG refers to the process of refining or reformulating user queries to enhance the retrieval module’s effectiveness. This step is critical in ensuring that the retriever fetches highly relevant and accurate documents or context from the knowledge base. Effective query rewriting includes disambiguating vague queries, expanding them with synonyms or related terms, correcting errors, and incorporating historical or contextual information. By improving the quality of the input to the retriever, query rewriting significantly boosts the overall performance of the RAG system, leading to more accurate and contextually appropriate outputs from the generation module. A user query can be transformed and decomposed in many ways before being executed as part of a RAG query engine, agent, or any other pipeline.

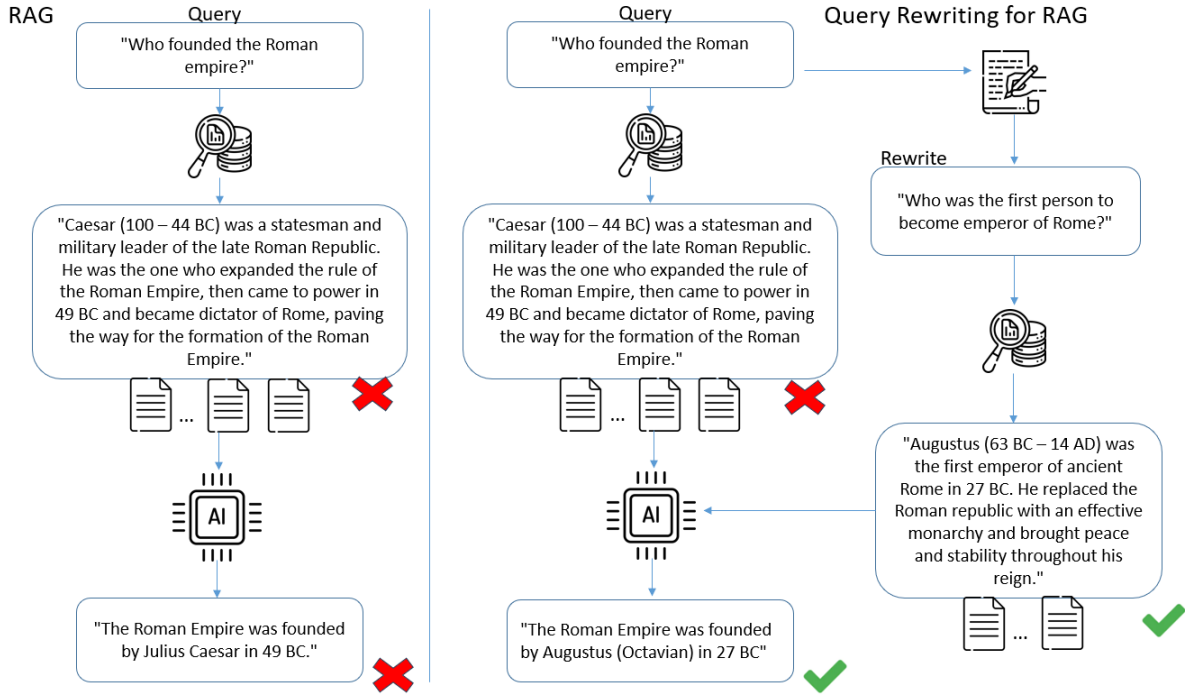


Figure 2.4: Query rewriting process in RAG

Embedding transformation: involves either pretraining language models or leveraging pretrained models to gain a deep understanding of the structural and semantic characteristics of text data. This process enables language models to map both queries and structured texts into one unified embedding space, facilitating efficient and accurate retrieval [28]. By doing so, the model enables efficient and accurate retrieval by ensuring that semantically related information from different formats is aligned within the same representation space. This unified embedding not only improves retrieval precision but also facilitates the integration of heterogeneous knowledge sources into the retrieval process.

2.1.5 Retrieval

Once the embedding process is complete, a document retrieval system identifies and returns the most relevant documents based on the query. In a RAG pipeline, this document retrieval step plays a crucial role, supplying the LLM with the appropriate documents for generating contextually accurate responses. Formally, given a query q in an arbitrary language x , it can retrieve document d in language y (which can be another language or the same language as x) from the corpus D^y . Moreover, the retrieval function $fn^*(\cdot)$ belongs to any of the functions: dense, lexical, or multi-vector retrieval [8].

$$d^y \leftarrow fn^*(q^x, D^y)$$

- **Dense retrieval**

Query q is transformed into the hidden states H_q based on a text encoder.

The normalized hidden state of the special token "[CLS]" is used for the representation of the query:

$$e_q = \text{norm}(H_q[0])$$

Similarly, the embedding of passage p is:

$$e_p = \text{norm}(H_p[0])$$

The relevance score between query and passage is measured by the inner product between the two embeddings e_q and e_p

$$s_{dense} \leftarrow \langle e_p, e_q \rangle$$

- **Lexical retrieval**

For each term $\mathbf{t}$ in the query:

$$w_{q_t} \leftarrow \text{ReLU}(W_{lex}^T H_q[i])$$

Similarly, for the passage:

$$w_{p_t} \leftarrow \text{ReLU}(W_{lex}^T H_p[i])$$

Where:

- $W_{lex} \in \mathbb{R}^{d \times 1}$: A weight matrix mapping the hidden state to a scalar.
- $H_q[i], H_p[i]$: Hidden states (embedding vectors) for the i -th token in the query and passage.
- ReLU: Rectified Linear Unit activation function, defined as $\text{ReLU}(x) = \max(0, x)$.

- **Multi-vector retrieval**

Representing query and passage with multiple vectors

Formulas:

- Query representation:

$$E_q = \text{norm}(W_{mul}^\top H_q)$$

- Passage representation:

$$E_p = \text{norm}(W_{mul}^\top H_p)$$

- $W_{mul} \in \mathbb{R}^{d \times d}$: A learnable projection matrix.
- H_q and H_p : Hidden states (embeddings) for the query and passage, respectively.
- norm: Column-wise normalization function applied to each vector in the matrices.

2.1.6 Post-retrieval

Reranking: The initial retrieval stage often relies on methods like dense or lexical retrieval, which are efficient but may not consistently yield highly relevant documents. To address this, reranking plays a crucial role by reorganizing the retrieved documents to better align with the given query, thereby improving their relevance.

Contextual compression: Contextual compression addresses the challenge of retrieving relevant information from large documents by compressing and filtering content based on the query. Instead of passing entire documents, which can lead to expensive and less effective responses, this approach refines the retrieved results. Using a base retriever and a document compressor tools like Compact [55], it will shortens documents by extracting only the relevant content or removing irrelevant ones entirely, ensuring more efficient and focused query responses.

2.2 Generator

The posed query and selected documents are synthesized into a coherent prompt to which a large language model is tasked with formulating a response. The model’s approach to answering may vary depending on task-specific criteria, allowing it to either draw upon its inherent parametric knowledge or restrict its responses to the information contained within the provided documents. In cases of ongoing dialogues, any existing conversational history can be integrated into the prompt, enabling the model to engage in multi-turn dialogue interactions effectively [14].

In this section, we provide an overview of the architecture of Transformer models [47] and delve into their core mechanisms that have revolutionized natural language processing. Following this, we explore various prompt engineering techniques, highlighting their role in optimizing the performance of pre-trained language models across a wide range of tasks, including RAG. These techniques not only improve the model’s understanding of task-specific contexts but also enhance its ability to generate accurate and contextually relevant responses.

2.2.1 Transformer architecture

Transformer [47], a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output [47]. It has revolutionized natural language processing (NLP) by enabling parallel processing of sequences. Unlike recurrent architectures such as RNNs [41] or LSTMs [17], which process inputs sequentially and struggle with long-range dependencies, Transformers use self-attention mechanisms to focus on relevant parts of the input across the entire sequence simultaneously. This approach not only improves the ability to capture intricate relationships within the data but also drastically increases computational efficiency by enabling systems to handle tasks in parallel. The result is a model architecture that excels at tasks requiring a deep understanding of context, from machine translation to text generation, setting the foundation for state-of-the-art language models like GPT [34], BERT [11], and T5 [36].

Encoder and decoder stacks: A transformer consists of two components: encoder and decoder. Each of them is a stack of N identical layers ($N=6$ in the original transformer model).

- **Encoder:** The encoder is composed of a stack of N identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. Around each of the two sub-layers, there is a residual connection [15] followed by layer normalization [2]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself [47].
- **Decoder:** The decoder, like the encoder, is also composed of a stack of N identical layers. In addition to the two sub-layers in each encoder layer, the encoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder Stack. The self-attention sub-layer in the decoder stack is modified to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i [47].

Positional encoding: plays a critical role in capturing the order of sequences, which is essential for preserving the structural integrity of text data. Positional encodings are added to the input embeddings at the initial layers of both the Encoder and Decoder stacks, enabling the model to incorporate sequential information.

In the original model, sine and cosine functions of different frequencies is used for positional encoding:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d_{\text{model}}}) \quad (2.1)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos(\text{pos}/10000^{2i/d_{\text{model}}}) \quad (2.2)$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid [47].

Self-attention mechanism:

- **Queries, keys and values:** In self-attention mechanism, transformer compute output by mapping a query with a set of key-value pairs. The output is computed as a weighted sum of the values, where the weight computed by scaled dot-product of query and keys.

- **Scaled-dot products:** The concept of self-attention lies in computing a weighted sum of values, where the weights represent the similarity between queries and keys. In Transformers, this similarity is determined using a dot product between the query and key vectors, followed by a softmax function. The softmax operation normalizes these weights, ensuring they fall within the range of 0 and 1, thus effectively controlling their influence.
- The vectors for queries, keys, and values are all of size d_K . When computing the dot product between queries and keys, the resulting values can become disproportionately large, especially as d_K increases. Such large values can cause gradients to vanish during backpropagation, hindering the training process. To mitigate this, transformers introduce a scaling factor $\sqrt{d_K}$, dividing the dot product by this value. This adjustment helps stabilize the gradients and ensures the training remains efficient and effective, even with large vector dimensions.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}V\right) \quad (2.3)$$

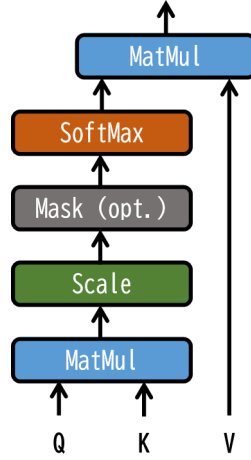


Figure 2.5: Scaled dot-product attention [47]

- **Multi-head attention:** Instead of performing a single attention function with keys, values and queries, it is good to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel. Here, $W_i^Q, W_i^K \in R^{d_{model} \times d_v}$, $W_i^Q \in R^{d_{model} \times d_v}$, $W^O \in R^{h d_v \times d_{model}}$ and h is the number of attention heads.

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \\ \text{where } \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned} \quad (2.4)$$

2.2.2 Prompt engineering

Prompt engineering is an essential methodology for maximizing the potential of pre-trained language models. It focuses on carefully crafting task-specific instructions, known as prompts, to direct the model's output effectively without requiring any modifications to its underlying parameters [43]. Various techniques have been developed to refine the practice of prompt engineering, each aimed at leveraging the full potential of pre-trained language models for diverse and complex applications. Among the most prominent methods are:

- **Zero-Shot (0S) prompting:** This technique involves providing the model with a clear and specific instruction for a task without any prior examples. This method provides maximum convenience, potential for robustness, and avoidance of spurious correlations [6]. Zero-shot prompting relies entirely on the model's pre-trained knowledge and ability to generalize across contexts, making it effective for a wide range of tasks without additional data.

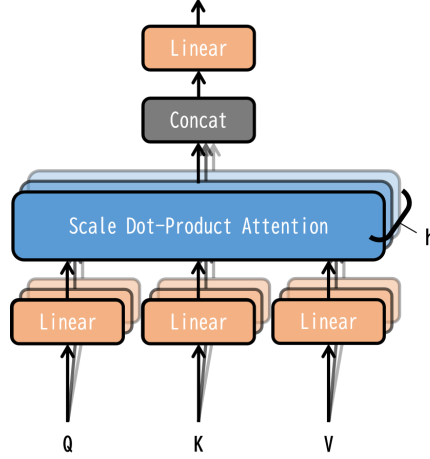


Figure 2.6: Multi-Head Attention (consists of several attention layers running in parallel) [47]

- **Few-Shot (FS) prompting:** This method works by giving K examples (with K often ranging from 10 to 100) of context and completion, and then one final example of context, then the model is expected to provide the final completion [6]. These K examples act as a guide for the model, helping it understand the structure, context, or desired behavior for the task.
- **One-Shot (1S) prompting:** This approach is the same as FS prompting but only one example is used for the prompt. This way of prompting closely matches the way in which some tasks are communicated to humans [6], normally we are given one single example or demonstration of the task then we are asked to complete other similar tasks.
- **Chain-of-Thought (CoT) prompting:** This technique involves prompting a coherent series of intermediate reasoning steps that lead to the final answer for a problem rather than providing a direct answer [53]. By simulating a logical progression of thought, the model is better equipped to handle complex tasks that require multi-step reasoning, such as mathematical problem-solving or logical deductions. This structured guidance fosters greater accuracy and depth in responses.

2.3 Augmentation methods

2.3.1 Augmentation stage: Inference

At the inference stage, augmentation involves retrieving and integrating external knowledge dynamically during the generation process. Recent work, such as in-context RALM [37], demonstrates the efficacy of prepending retrieved documents to the input text without modifying the language model (LM) architecture. This approach leverages off-the-shelf retrievers for document selection, providing significant performance gains while maintaining simplicity.

Language models define probability distributions over sequences of tokens. Given such a sequence $x_1, \dots, x_n$, the standard way to model its probability is via next-token prediction:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_{<i}),$$

where $x_{<i} := x_1, \dots, x_{i-1}$ is the sequence of tokens preceding x_i , also referred to as its prefix. This autoregressive model is usually implemented via a learned transformer network [47] parameterized by the set of parameters θ :

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p_{\theta}(x_i | x_{<i}).$$

Retrieval-augmented language models (RALMs) add an operation that retrieves one or more documents from an external corpus C , and conditions the above LM predictions on these documents. Specifically, for predicting x_i , the retrieval operation from C depends on its prefix $RC(x_{<i})$, giving rise to the general RALM decomposition:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_{<i}, RC(x_{<i})).$$

In-context RALM refers to a simple method of concatenating the retrieved documents within the transformer’s input before the prefix, without altering the LM weights θ :

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p_\theta(x_i | [RC(x_{<i}); x_{<i}]),$$

where $[a; b]$ denotes the concatenation of strings a and b . This approach allows the model to incorporate external knowledge efficiently.

Two practical design choices in RALM systems significantly impact performance:

1. **Retrieval stride:** Retrieval operations can be performed once every $s > 1$ tokens to reduce computation costs. The retrieval stride s determines how often new documents are fetched, trading runtime for performance. The formulation becomes:

$$p(x_1, \dots, x_n) = \prod_{j=0}^{\lfloor n/s \rfloor - 1} \prod_{i=1}^s p_\theta(x_{s \cdot j + i} | RC(x_{\leq s \cdot j}); x_{<(s \cdot j + i)}).$$

2. **Retrieval query length:** The retrieval query at stride j is typically restricted to the last ℓ tokens of the prefix ($p_j^{s, \ell} := x_{s \cdot j - \ell + 1}, \dots, x_{s \cdot j}$) to avoid diluting the relevance of the information. The resulting formulation is:

$$p(x_1, \dots, x_n) = \prod_{j=0}^{\lfloor n/s \rfloor - 1} \prod_{i=1}^s p_\theta(x_{s \cdot j + i} | [RC(q_j^{s, \ell}); x_{<(s \cdot j + i)}]).$$

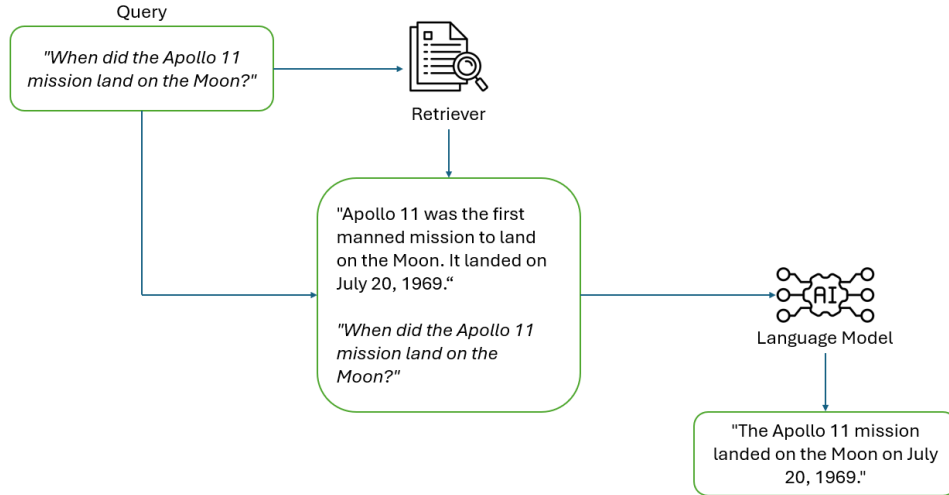


Figure 2.7: Augmentation during the inference stage. Relevant documents are retrieved and prepended to the input for improved factual grounding

2.3.2 Augmentation source: Structured data

Structured data, such as knowledge bases (KBs), offers a rich source for augmentation. The KnowledgeGPT framework [51] demonstrates how LLMs can interact with KBs for both retrieval and storage. By generating programmatic queries, LLMs can navigate multi-hop and entity-specific questions effectively.

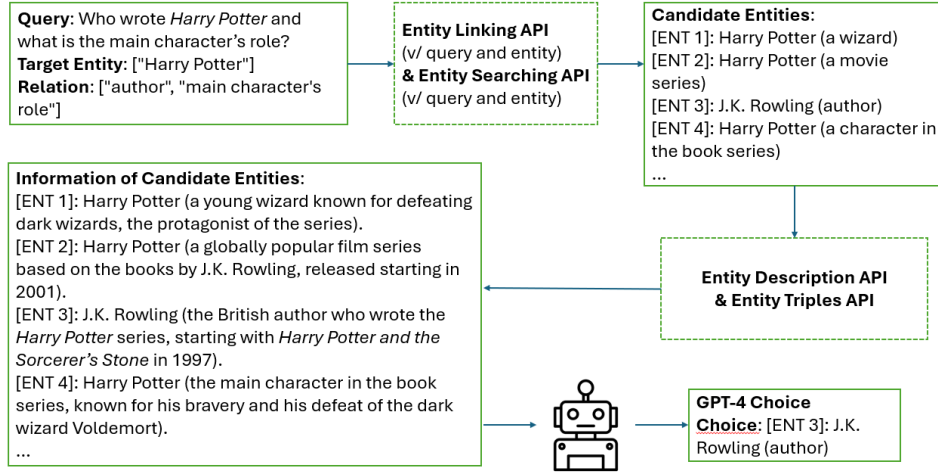


Figure 2.8: Workflow of augmentation with structured data sources [51]

To enable effective interaction with structured data, KnowledGPT employs several key functions:

- *get_entity_info*: Retrieves encyclopedic descriptions of an entity from the KB.
- *find_entity_or_value*: Extracts entities or attribute values related to a query by identifying the most relevant relation.
- *find_relationship*: Identifies relationships between two entities within the KB.

These functions allow KnowledGPT to leverage the structured nature of KBs to extract precise and relevant knowledge.

The *find_entity_or_value* function is central to querying entities and attribute values. It matches the query's relation to KB triples using embedding-based similarity and retrieves the corresponding entities or values. The algorithm is detailed below.

2.3.3 Augmentation process

The augmentation process in RAG frameworks can be categorized into various types, such as once, iterative, recursive, and adaptive. Among these, the Self-reflective retrieval-augmented generation (SELF-RAG) [1] approach employs an adaptive process where the model dynamically determines the necessity of retrieval, evaluates the relevance and support of retrieved passages, and critiques its own output for factuality and quality.

Reflection tokens are special tokens used by SELF-RAG to evaluate and guide the retrieval and generation process.

Type	Input	Output	Definitions
Retriever	$x/x, y$	{yes, no, continue}	Decide to Retrieve with $\mathcal{R}$
IsREL	x, d	{ relevant , irrelevant }	d provide useful information to solve x
IsSUP	x, d, y	{ fully supported , partially supported, no support}	All of the verification-worthy statement in y is supported by d
IsUSE	x, y	{5, 4, 3, 2, 1}	y is useful response to x

Table 2.1: Reflection tokens [1]

The inference process in SELF-RAG leverages reflection tokens to dynamically retrieve relevant passages and critique the model's outputs. The steps are as follows:

1. **Input and prediction:** The model receives the input prompt x and preceding generation $y < t$. It predicts whether retrieval is necessary using the retrieve token.
2. **Retrieve:** If retrieval is needed (retrieve = yes), the retriever fetches relevant passages D based on the input and context.
3. **Generate and evaluate:** For each retrieved passage $d \in D$, the model:

Algorithm 6 find_entity_or_value

Input: query q , alias list of entity $\mathcal{E}$, alias list of relation $\mathcal{R}$.

Output: a list of target entities or attribute values $\mathcal{T}$.

```
1: function EMBSIM(str  $r$ , list[str]  $\mathcal{R}$ )
2:    $s = -1$ 
3:    $r = \text{embedding}(r)$ 
4:   for  $r_i \in \mathcal{R}$  do
5:      $r_i = \text{embedding}(r_i)$ 
6:     if  $\cos(r, r_i) > s$  then
7:        $s = \cos(r, r_i)$ 
8:     end if
9:   end for
10:  return  $s$ 
11: end function
12:  $e = \text{entity\_linking}(\mathcal{E})$ 
13: if  $e == \text{NULL}$  then
14:   return  $\text{NULL}$ 
15: end if
16:  $r = \text{NULL}$ 
17:  $s_r = -1$ 
18: for  $\text{triple} \in \text{triples}$  do
19:    $r_i = \text{triple.rel}$ 
20:    $s_i = \text{EMBSIM}(r_i, \mathcal{R})$ 
21:   if  $s_i > s_r$  then
22:      $s_r = s_i$ 
23:      $r = r_i$ 
24:   end if
25: end for
26: if  $r == \text{NULL}$  then
27:   return  $\text{NULL}$ 
28: end if
29:  $\text{triples}_r = \text{triples}$  with relation  $r$ 
30:  $\mathcal{R} =$  target entities or attribute values in  $\text{triples}_r$ 
31: return  $\mathcal{R}$ 
```

- Predicts IsREL to assess relevance.
 - Generates the next segment y_t .
 - Predicts IsSUP and IsUSE to evaluate support and utility.
4. **Rank and select:** The model ranks all generated segments based on IsREL, IsSUP, and IsUSE, and selects the top-ranked segment.
 5. **No retrieval case:** If retrieval is not needed (Retrieve = no), the model directly generates the next segment and predicts its utility.
 6. **Iterative refinement:** The process repeats until the response is complete.

Algorithm 7 SELF-RAG inference

Require Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \dots, d_N\}$

- 1: **Input:** input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
 - 2: $\mathcal{M}$ predicts Retriever given $(x, y_{<t})$
 - 3: **if** Retriever == Yes **then**
 - 4: Retrieve relevant text passages $\mathcal{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ Retrieve
 - 5: $\mathcal{M}$ predicts IsREL given x, d and $y_{<t}$ for each $d \in \mathcal{D}$ ▷ Generate
 - 6: $\mathcal{M}$ predicts IsSUP and IsUSE given x, d for each $d \in \mathcal{D}$ ▷ Critique
 - 7: Rank y_t based on IsREL, IsSUP, IsUSE
 - 8: **else if** Retriever == No **then**
 - 9: $\mathcal{M}_{\text{gen}}$ predicts y_t given x ▷ Generate
 - 10: $\mathcal{M}_{\text{gen}}$ predicts IsUSE given x, y_t ▷ Critique
 - 11: **end if**
-

3.1 RAG and LLMs in coding tasks

3.1.1 Retrieval strategies

Sparse retrieval methods: Tools like BM25 [5] rank documents based on term frequency and inverse document frequency (TF-IDF). While highly effective for small-scale knowledge bases, their efficiency declines as data scales increase [16].

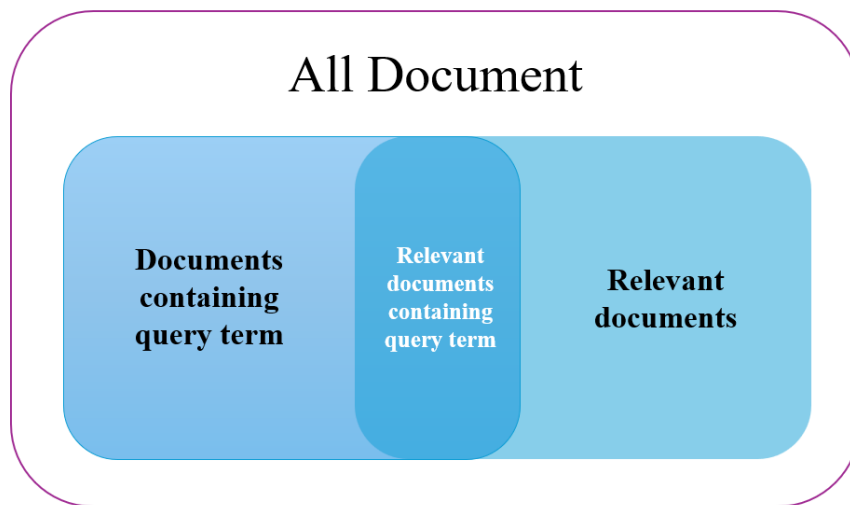


Figure 3.1: BM25 visualization

Dense retrieval methods: Leveraging semantic embeddings, dense retrieval tools like SBERT [38]’s semantic search and approximate techniques (e.g., HNSW [29], ANNOY [4], and LSH [21]) provide scalable solutions for large knowledge bases. These approaches achieve significant speed improvements with minimal accuracy trade-offs [16].

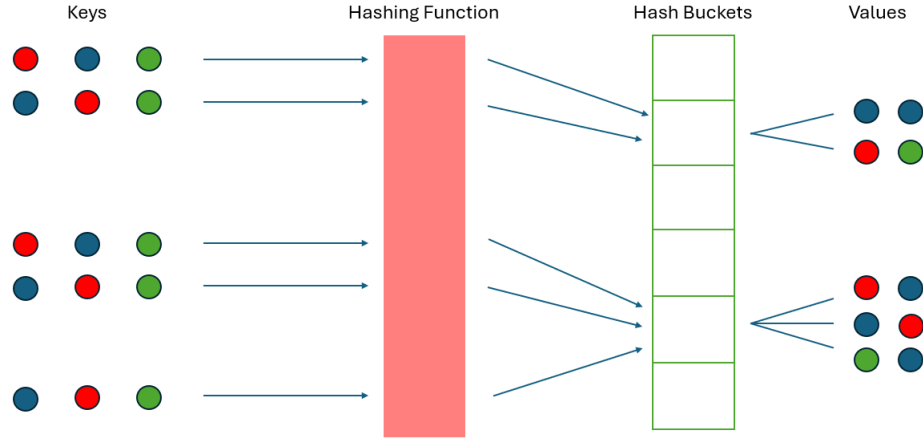


Figure 3.2: LSH visualization

Selective retrieval: Frameworks such as Repoformer [54] employ selective retrieval, determining the necessity of retrieval based on contextual evaluation. This approach reduces inefficiencies and avoid adding irrelevant context.

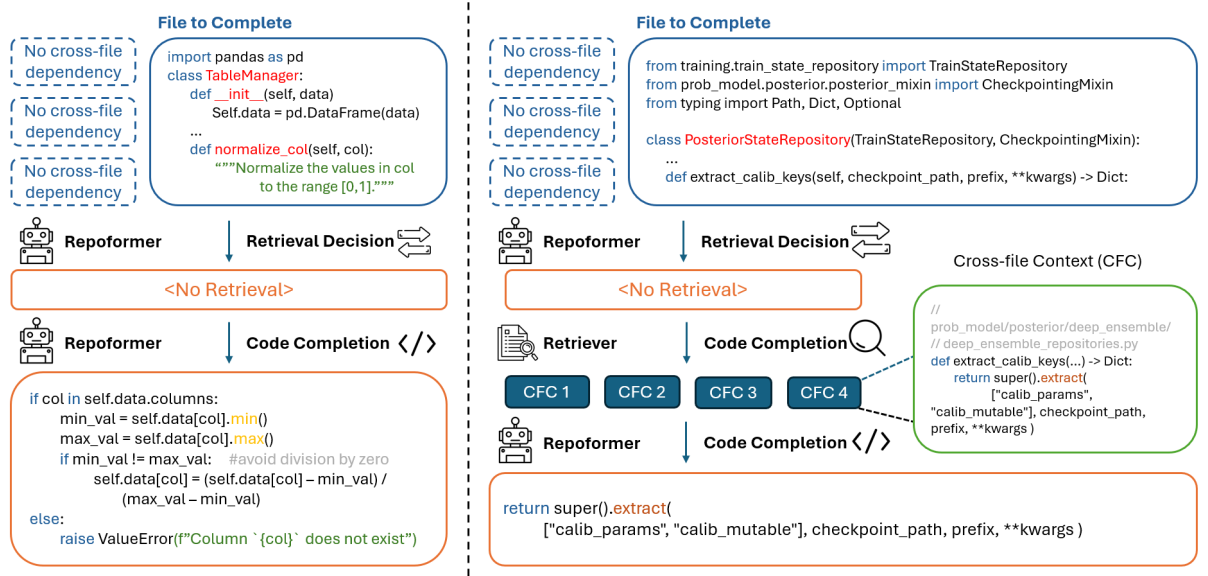


Figure 3.3: REPOFORMER framework [54]

Multi-retrieval perspectives: Systems like ProCC [45] utilize diverse perspectives, including lexical semantics, hypothetical line embeddings, and code summarization, to enrich context and improve code completion accuracy.

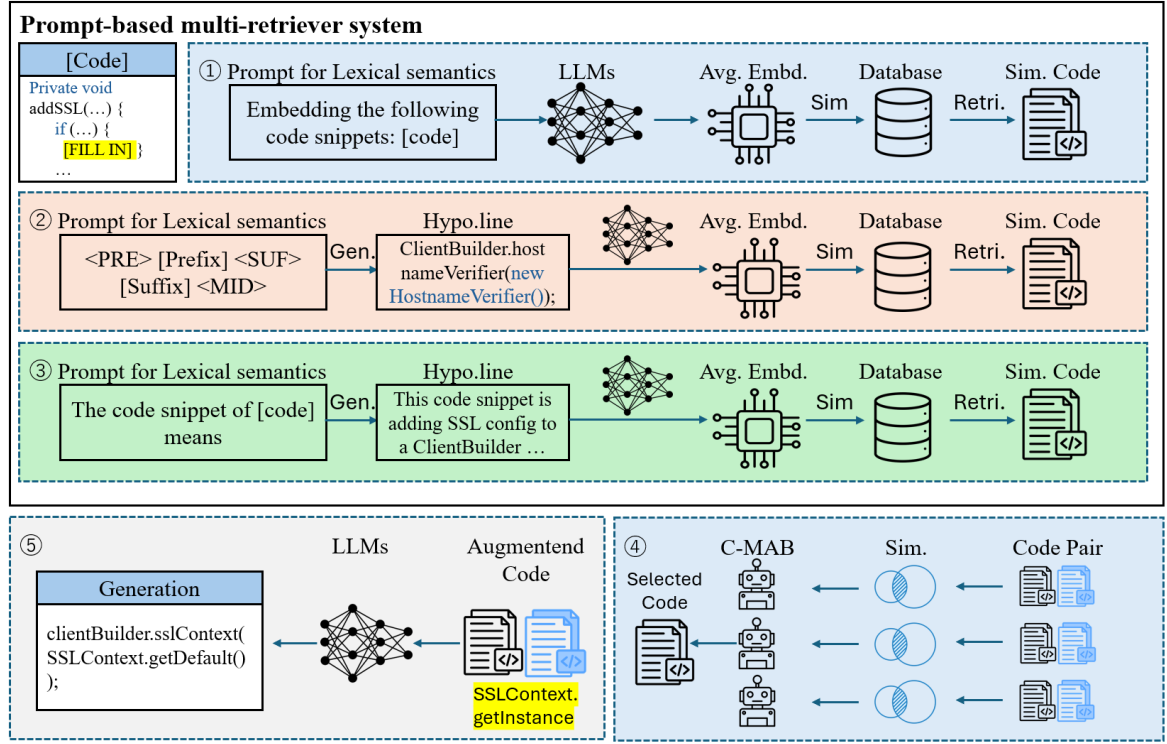


Figure 3.4: ProCC framework [45]

Multilingual and multifunctional embeddings: Models like BGE M3-Embedding [8] support over 100 languages and various retrieval functionalities, including dense, multi-vector, and sparse retrieval, enhancing versatility in coding tasks.

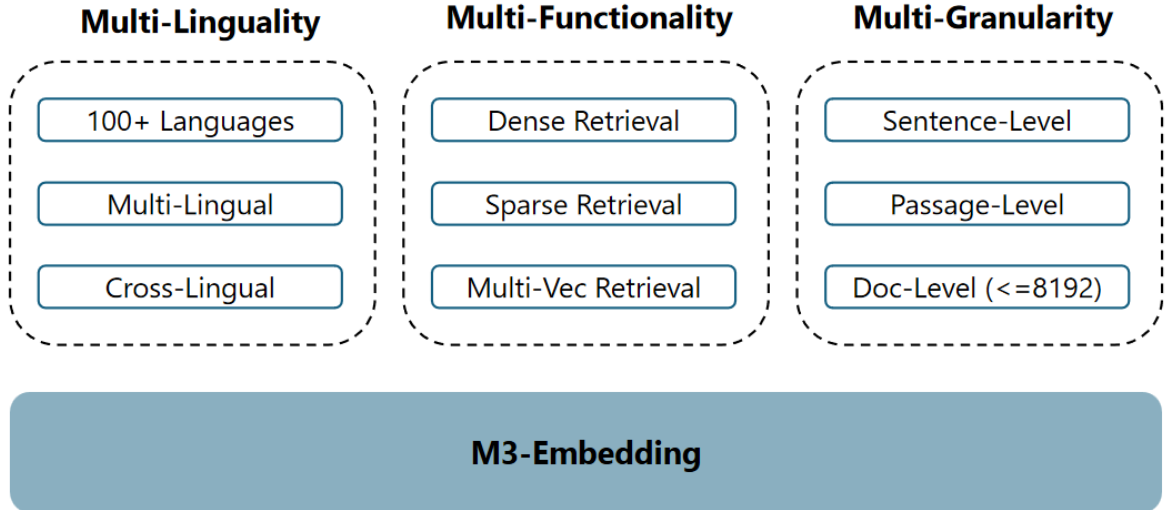


Figure 3.5: Characters of M3-Embedding [8]

3.1.2 Generative enhancements

Generative Pre-trained Transformers: LLMs like GPT-2 [35] and code Llama [40] are trained on extensive code corpora, enabling them to handle diverse tasks such as function completion and API usage.

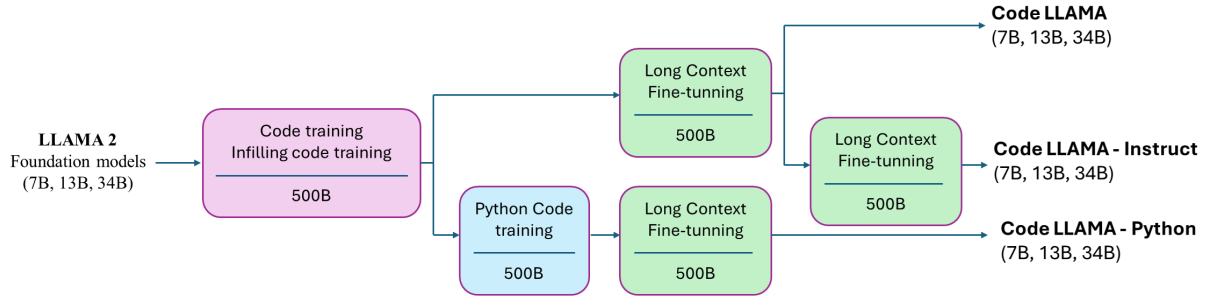


Figure 3.6: The Code Llama specialization pipeline [40]

In-context augmentation: RAG pipelines integrate retrieved code snippets directly into prompts, enhancing generation without additional fine-tuning. This approach is particularly effective for repository-level code generation [18].

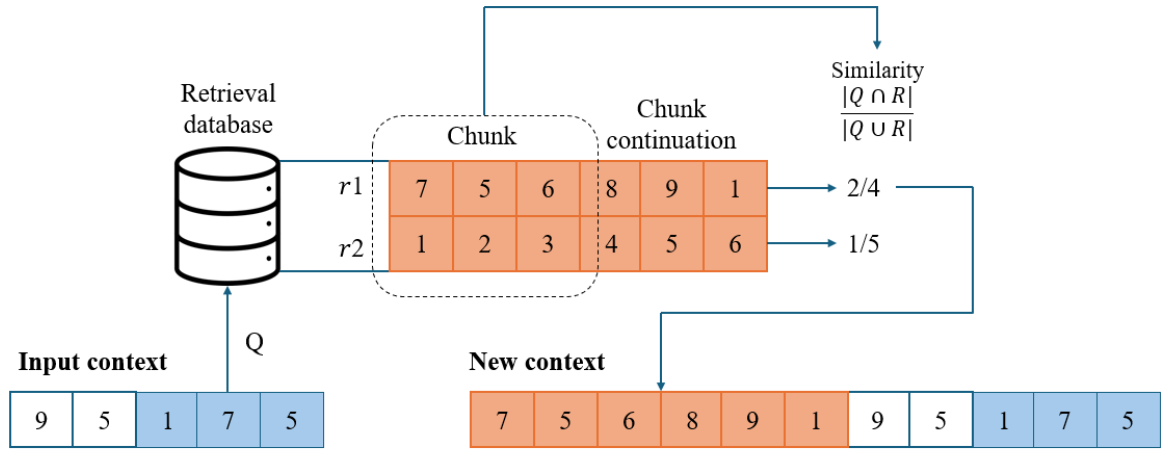


Figure 3.7: The retrieved snippet contains continuation tokens and additional metadata [18]

Specialized models for coding: Models like Qwen2.5-Coder [19] and CodeQwen1.5-7B-Chat [3] are fine-tuned for coding tasks, demonstrating improved performance in code generation and comprehension.

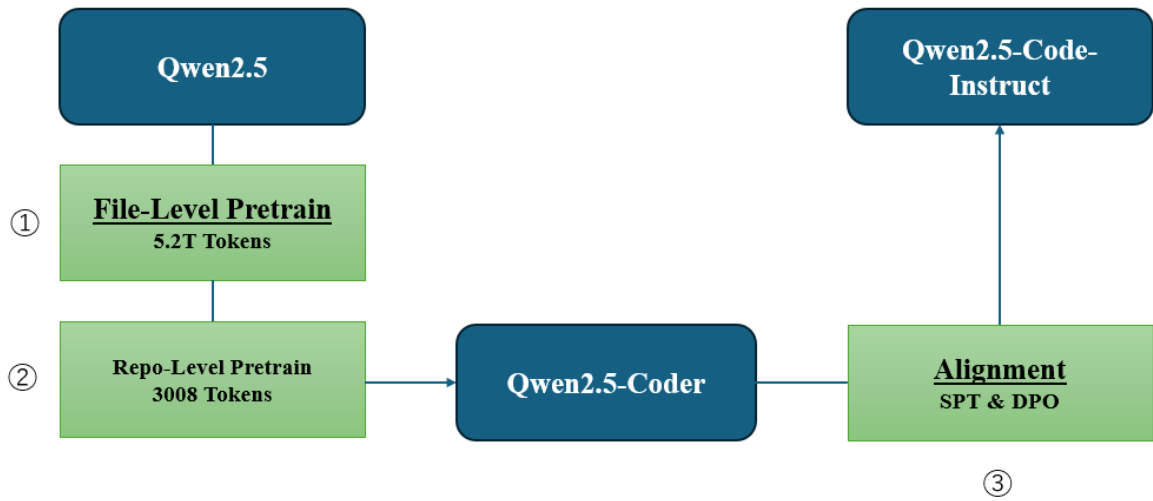


Figure 3.8: The three-stage training pipeline for Qwen2.5-Coder [18]

3.1.3 Specialized Applications

Repository-level code completion: Tools like Repoformer and RepoCoder [56] focus on cross-file dependencies, retrieving and integrating relevant code snippets to improve repository-wide coherence.

<pre># Below are some referential code fragments # from other files: # ----- # the below code fragment can be found in # tests/test_pipelines_common.py # ----- # @unittest.skipIf(torch_device != "cuda") # def test_to_device(self): # components = self.get_dummy_components() # pipe = self.pipeline_class(**components) # pipe.progress_bar(disable=None) # pipe.to("cpu") # ----- """Based on above, complete the following code:"""</pre>	# Retrieved Code
<pre>@unittest.skipIf(torch_device != "cuda") def test_float16_inference(self): components = self.get_dummy_components()</pre>	# Unfinished Code
<pre>pipe = self.pipeline_class(**components) pipe.to(torch_device)</pre>	# Model Prediction

Figure 3.9: Demonstrating the format of the RepoCoder prompt [56]

Commit message and assertion generation: RAG systems generate commit messages and assertions by retrieving semantically aligned examples, balancing efficiency and effectiveness [16].

Code hallucination mitigation: Automated code generation, powered by LLMs, has significantly enhanced development efficiency by generating code from input requirements. However, LLMs often produce hallucinations, particularly when handling complex contextual dependencies in repository-level development scenarios. To address these challenges, RAG-based mitigation methods have been proposed [59].

3.2 Developer assistance tools

3.2.1 Functionality

Search and Retrieval Tools: Platforms like stack overflow¹ provide a database of Q&A posts where users search for solutions based on keywords or queries. Neural code search (NCS) [42] enhances this by enabling natural language queries over raw codebases without curated answers, using vector embeddings and ranking strategies.

¹Stack Overflow - <https://stackoverflow.com/>

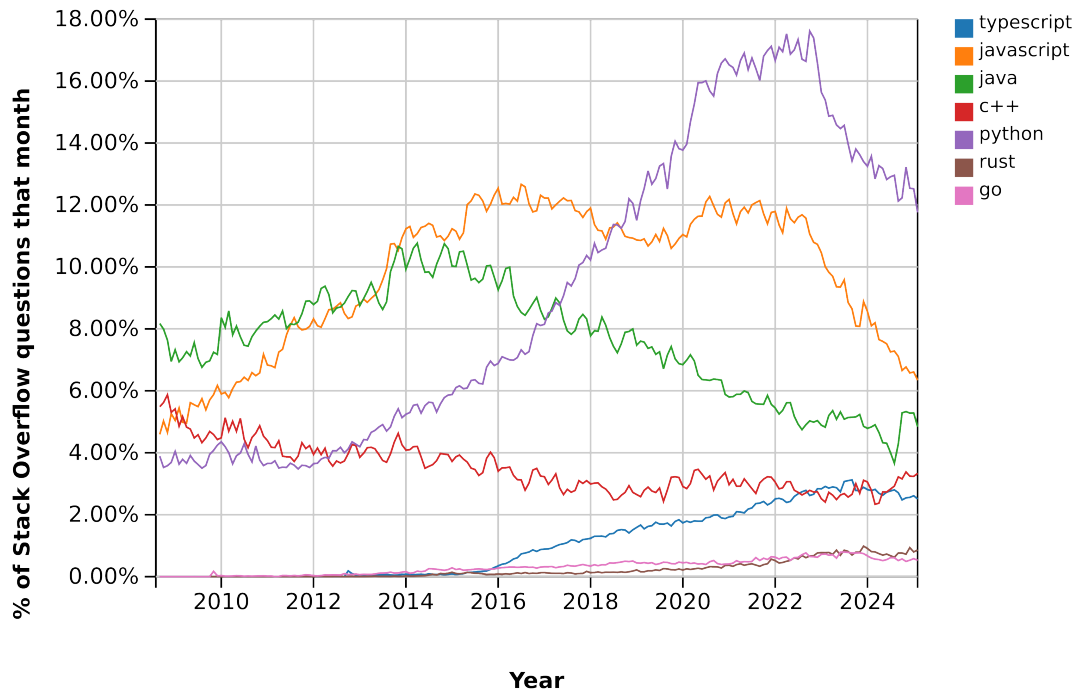


Figure 3.10: Stackoverflow trends chart 2008-2025

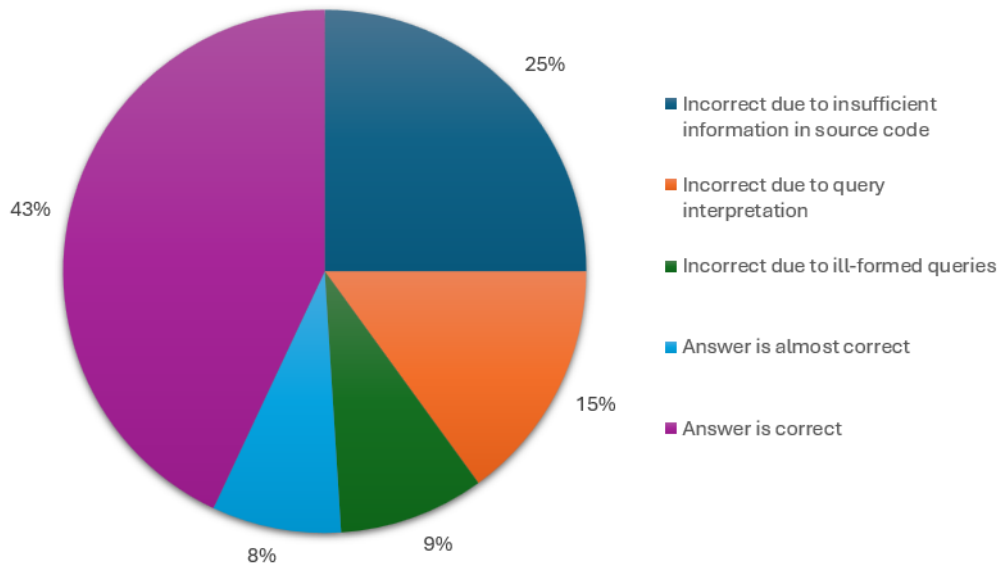


Figure 3.11: NCS evaluation results [42]

Code completion and suggestion systems: Intelligent tools like IntelliCode Compose [44] and example-based systems [7] prioritize providing relevant suggestions for method calls, arguments, and entire lines of code. These systems are designed to enhance developer productivity by learning from existing codebases.

Hybrid generation systems: RAG models combine retrieval with generative capabilities [25], producing specific and contextually appropriate outputs by accessing external document repositories and leveraging models

3.2.2 Retrieval mechanisms

Keyword-based retrieval: Traditional forums rely on keyword searches, often yielding broad results with low contextual relevance.

Embedding-based retrieval: Tools like NCS employ word embeddings, TF-IDF weighting [31], and vector similarity for precise retrieval of relevant code snippets. These systems also incorporate syntactic and semantic elements extracted from source code.

Rule-based retrieval: Example-based systems mine patterns and associations (e.g., frequent co-occurrence of API calls) from repositories to recommend contextually relevant options.

3.2.3 Code generation capabilities

Static suggestion systems: Earlier systems [7] relied solely on static type information, offering alphabetically sorted method recommendations without understanding contextual relevance.

Generative models: IntelliCode Compose represents a generative approach, using transformer-based architectures [47] like GPT-C to predict and generate entire lines or blocks of syntactically correct code. This includes multilingual support and optimized decoding methods like beam search.

3.2.4 Integration in development processes

Standalone search interfaces:

- **NCS** [42]: Functions as an external platform where developers input natural language queries to retrieve relevant code snippets directly from large codebases.
- **Sourcegraph**²: Provides a universal code search and navigation tool, enabling developers to search across multiple repositories and languages from a centralized interface.
- **Krugle**³: A specialized search engine designed for searching open-source and enterprise code repositories. It indexes and organizes code by context, allowing developers to locate relevant examples, API usage, and documentation efficiently.

Feature	NCS	Sourcegraph	Krugle
Functionality	Retrieves code snippets from large codebases using natural language queries.	Provides universal code search and navigation across multiple repositories and languages.	Searches open-source and enterprise code repositories, indexing code by context.
Search methodology	Maps natural language queries and code snippets into a shared vector space using neural networks.	Offers structural code search, matching nested expressions and code blocks beyond regular expressions.	Indexes code with advanced filtering for examples, API usage, and documentation.
Supported languages	Initially focused on Java.	Supports over 30 programming languages.	Supports multiple programming languages.
Deployment options	Research project with datasets available for evaluation.	Offers self-hosted solutions via docker compose and kubernetes, as well as a managed cloud service.	Web-based platform.

Table 3.1: Comparison of NCS, Sourcegraph, and Krugle

IDE-embedded assistance:

- **IntelliCode Compose** [44]: Integrates into VS Code, offering real-time code generation and suggestions.
- **Github Copilot**⁴: Embedded within various IDEs, including VS Code and JetBrains, it leverages AI to provide code completions and suggestions based on the current context.
- **Tabnine**⁵: An AI-powered code completion tool that integrates with various IDEs, offering whole-line and full-function code completions based on the current code context.

²Sourcegraph - <https://sourcegraph.com/>

³Krugle - <https://www.krugle.com/>

⁴Github Copilot - <https://github.com/features/copilot>

⁵Tabnine - <https://www.tabnine.com/>

Feature	IntelliCode Compose	GitHub Copilot	Tabnine
Functionality	Provides real-time code generation and suggestions within VS Code.	Offers AI-driven code completions and suggestions across various IDEs.	Delivers whole-line and full-function code completions based on current code context.
Supported IDEs	VSCode.	VSCode, JetBrains suite, Neovim, and others.	VSCode, IntelliJ IDEA, Sublime Text, Atom, and more.
AI model	Utilizes machine learning models trained on open-source GitHub repositories.	Powered by OpenAI's Codex model, trained on a vast dataset of public code.	Employs proprietary models trained on permissively licensed open-source code.
Privacy and security	Processes code locally, enhancing privacy.	Processes code in the cloud, which may raise privacy concerns for sensitive codebases.	Offers both cloud-based and on-premises solutions, with options for local processing to ensure code privacy.
Pricing	Included with VS Code.	Individual model: \$10/month; Business version at \$19/month and Enterprise at \$39/user/month	Basic model: free version with basic features; Dev version at \$9/month and Enterprise at \$39/user-month

Table 3.2: Comparison of IntelliCode Compose, GitHub Copilot, and Tabnine

```

1 # From: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
2 import torch
3 import torchvision
4 import torchvision.transforms as transforms
5
6 transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5, 0.5,
7                                     0.5), (0.5, 0.5, 0.5))])
8
9 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
10                                         download=True, transform=transform)
11 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
12                                           shuffle=True, num_workers=2)
13
14 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
15                                         download=True, transform=transform)
16 testloader = torch.utils.data.DataLoader(testset, batch_size=1, shuffle=False,
17                                          num_workers=2)

```

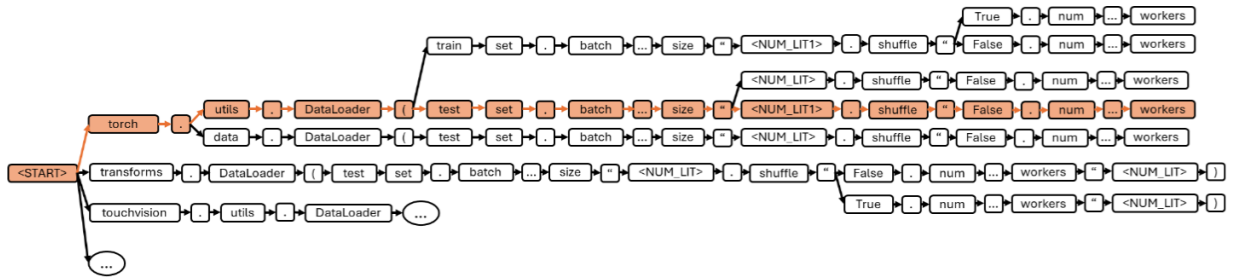


Figure 3.12: IntelliCode Compose’s code snippet completion suggestion and completion-tree demonstration [44]

4.1 Fine-tuning vs. RAG

4.1.1 Fine-tuning

Fine-tuning involves adapting a pre-trained LLM to specific tasks or domains by further training it on a targeted dataset. This process refines the model's parameters to align with specialized requirements, enhancing its performance in particular applications.

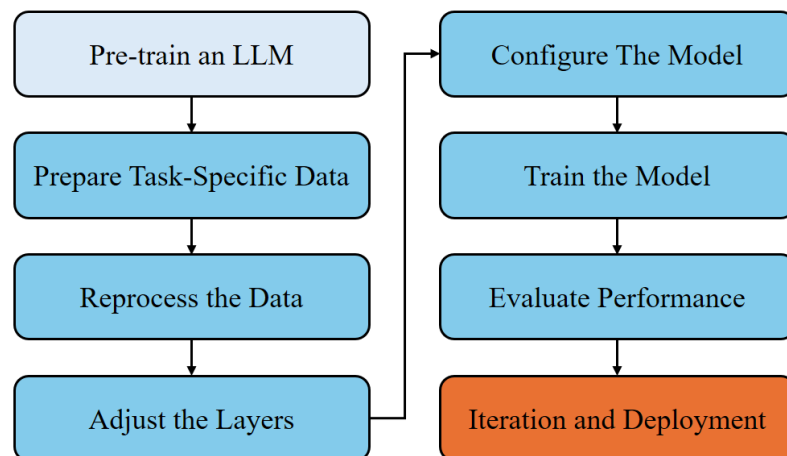


Figure 4.1: Fine-tuning process¹

¹RAG vs Fine-Tuning: A Comparative Analysis of LLM Learning Techniques - <https://addepto.com/blog/rag-vs-fine-tuning-a-comparative-analysis-of-llm-learning-techniques/>

4.1.2 Comparison between fine-tuning and RAG

Aspect	Fine-tuning	RAG
Dynamic vs. Static	Static: Relies on pre-trained knowledge, requiring retraining to update information.	Dynamic: Accesses up-to-date external data sources, providing current information without retraining.
Architecture	Utilizes a pre-trained LLM adjusted with task-specific datasets.	Combines LLM with external knowledge bases, enabling retrieval and generation.
Customization	High customization in domain-specific knowledge, tone, and style.	Limited customization to domain behavior but excels in integrating external content.
Hallucinations	Reduces hallucinations through domain-specific training but may hallucinate with unknown queries.	Minimizes hallucinations by grounding responses in retrieved documents.
Accuracy	High accuracy in specific, trained domains but limited outside the training scope.	Highly accurate for retrieving up-to-date, domain-relevant information.
Transparency	Functions as a "black box" with limited insight into response reasoning.	Transparent response generation with traceable data sources.
Cost	Higher cost due to significant computational resources, labeled data requirements, and periodic retraining for updates.	Lower cost as it eliminates the need for retraining and requires minimal or no labeled data by leveraging external knowledge bases.
Complexity	More complex due to the need for expertise in NLP, deep learning, and hyperparameter tuning.	Simpler to set up with minimal coding for retrieval integration.
Adaptability	Less adaptable, requiring retraining for new information.	Easily adapts to new data and dynamic environments.

Table 4.1: Comparison of fine-tuning and RAG

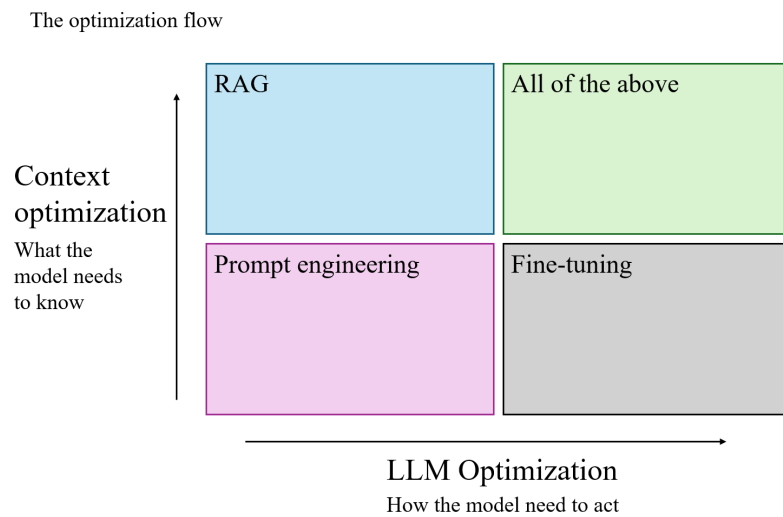


Figure 4.2: The nature of fine-tuning and RAG²

4.1.3 Rationale for choosing RAG

Minimize hallucinations: RAG provides real-time access to the latest documentation and programming guidelines, ensuring that code suggestions are current and precise without the need for extensive retraining.

²Fine-tuning vs. RAG: Understanding the Difference - <https://finetunedb.com/blog/fine-tuning-vs-rag/>

Transparency: By grounding responses in specific, retrievable documents, RAG offers clear traceability of information sources. This transparency fosters trust in the generated outputs, as developers can verify the origins of the suggestions provided.

4.2 Why BGE-M3?

In RAG systems, the quality of embeddings plays a critical role in determining the relevance and precision of retrieved content. BGE-M3 [8], a state-of-the-art embedding model, has demonstrated superior performance in multilingual retrieval tasks, as shown in Table 4.2. Compared to baseline models such as BM25 [39], mDPR [58], mContriever [20], mE5_{large} [50], E5_{mistral-7b} [49], BGE-M3 achieves an average retrieval score of 71.5 on the MIRACL dataset [57], significantly outperforming prior methods. This score reflects its advanced ability to generate robust and semantically meaningful representations.

The strength of BGE-M3 lies in its ability to unify dense retrieval, sparse retrieval, and multi-vector retrieval techniques. This integration allows it to handle diverse query types with high accuracy and flexibility. Notably, its performance exceeds that of other models in the M3-Embedding framework, with its "All" configuration achieving the highest score by combining the strengths of different retrieval approaches.

In our RAG pipeline, BGE-M3 serves as the foundation for the retrieval stage, ensuring that only the most relevant and semantically precise information is retrieved from the vector database. By leveraging BGE-M3’s advanced capabilities, we aim to enhance the overall effectiveness of the RAG system by improving the quality and relevance of the retrieved context, thus contributing to more accurate and contextually appropriate responses.

Model	Avg Score
Baselines	
BM25 [39]	31.9
E5 _{mistral-7b} [49]	63.4
OpenAI-3	54.9
M3-Embedding [8]	
Dense	69.2
Sparse	53.9
Multi-vector	70.5
Dense+Sparse	70.4
All	71.5

Table 4.2: Comparison of BGE-M3 embeddings with baseline models for multi-lingual retrieval performance on the MIRACL dataset [8]

4.3 Why Ollama’s LLMs

In designing a fully local RAG system, Ollama was selected as the LLM runtime for its ease of use, lightweight architecture, and compatibility with a wide range of open-source models. Ollama provides a simple yet powerful API interface for serving large language models directly on local machines, removing the dependency on cloud-based inference and ensuring full control over data privacy and latency. Its support for models such as Qwen, LLaMA, and Mistral makes it a versatile backend for experimentation and deployment. Moreover, the CLI-based installation and Docker integration simplify the setup process, aligning with the goal of being developer-friendly and easy to deploy. By utilizing Ollama, our system guarantees that all generation processes remain offline, making it suitable for privacy-conscious environments and offline development workflows.

4.4 Why Docker?

Docker was chosen as the containerization solution for our RAG system to ensure portability, consistency, and ease of deployment across various development environments. Our backend—comprising the retrieval service, embedding logic, and integration with the vector database—is relatively complex and includes multiple dependencies and components that can be challenging to configure manually. Docker

provides a robust solution to package these services into isolated containers, enabling users to launch the full backend stack with a single command, regardless of their underlying operating system or local environment setup.

By using Docker, we abstract away the intricacies of installing and configuring backend tools such as the Milvus vector database and the application logic for hybrid retrieval. This not only simplifies onboarding for new users but also ensures that the system behaves consistently across machines, eliminating common "it works on my machine" problems. Additionally, Docker's support for volume mounting and environment variables offers flexibility in managing different versions of data, models, or configurations without altering the core codebase.

Although the backend is not lightweight due to the nature of the services it includes, containerization allows the system to remain modular and maintainable. Users can deploy updates or scale specific services independently, without affecting the rest of the system. This separation of concerns aligns with modern software engineering best practices and makes the system easier to extend and maintain in the long term.

4.5 Why VS Code Extension?

Integrating the RAG system into a Visual Studio Code extension provides seamless, in-editor support for developers within their preferred development environment. This integration enables users to query documentation, generate code, and receive intelligent suggestions in real time—without needing to switch between tools or leave the editor. By minimizing context-switching, it streamlines the development workflow and boosts productivity. Visual Studio Code's extensibility, widespread usage in the web development community, and robust support for terminals and custom interfaces make it an ideal platform for embedding such a developer-focused RAG solution.

4.6 Main architecture

Our system, called RAGGIN (Retrieval-Augmented Generation for Guided Intelligence in Next.js), is designed to enhance the performance and contextual understanding of large language models by leveraging external knowledge sources. The system architecture is composed of the following key components:

- **Vector database:** Stores the Next.js documentation in an optimized vector format to enable efficient semantic retrieval.
- **Retriever:** Accepts the user query sent from the VS Code extension and retrieves the most relevant documents from the vector database.
- **Augmentation:** Combines the user query and retrieved documents to construct a context-rich prompt, including relevant context and additional questions, and forwards it to the generator.
- **Generator:** Receives the augmented prompt and utilizes the local LLM to generate accurate and contextually relevant responses.

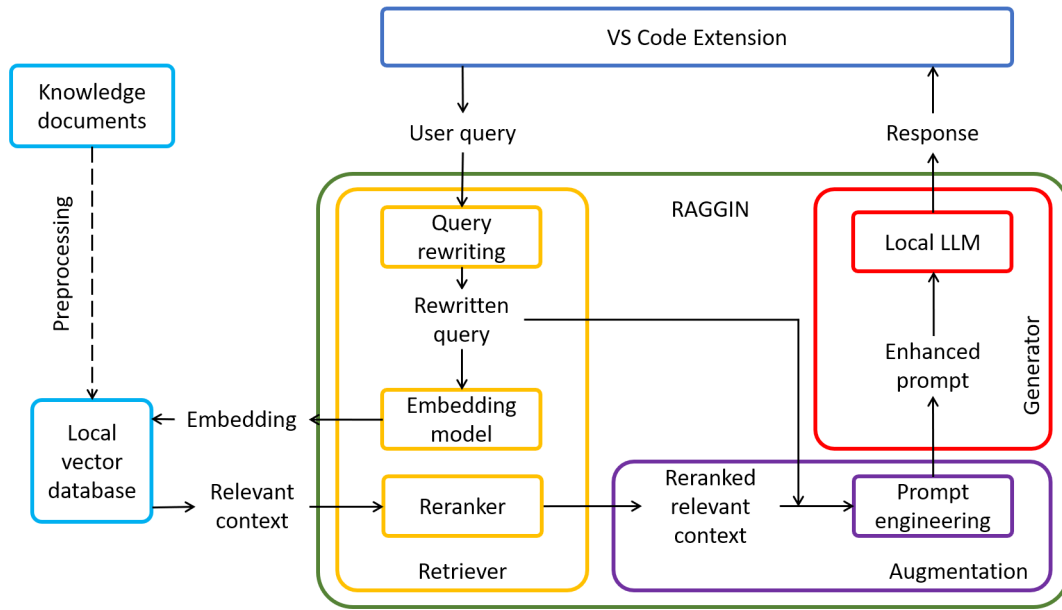


Figure 4.3: RAGGIN architecture design

In this architecture, the input is the user's query, and the output is the corresponding response generated by the system. The response is constructed by utilizing the retrieved data and contextual knowledge extracted from the Next.js documentation, ensuring that it is both accurate and relevant to the user's query.

How the RAGGIN system solves the problems

The proposed RAGGIN system is designed to address the key issues outlined in the problem statement by combining advanced retrieval and generation techniques with seamless integration into the developer workflow. Below is how the system resolves the identified problems:

1. Time-consuming searches

Problem: Developers spend excessive time manually searching through forums like StackOverflow or through vast and scattered Next.js documentation to find specific information.

Solutions:

- The retriever component of the system uses a vector database to store Next.js documentation in a semantically indexed format.
- When a user submits a query, the retriever efficiently retrieves the most relevant documents based on semantic similarity
- By narrowing down the search scope and returning only the most contextually relevant sections, the system reduces the time spent on manual searches and ensures higher accuracy in retrieval.

2. Lack of context-aware assistance

Problem: Retrieved information often lacks sufficient context, leaving developers to manually piece together fragmented answers.

Solutions:

- The augmentation component combines the retrieved documents with the user's query to construct a prompt enriched with relevant context.
- The generator uses the augmented prompt to produce coherent, actionable, and contextually accurate responses. This eliminates the need for developers to manually correlate information from multiple sources.

3. Workflow disruption

Problems: Switching between the code editor and external documentation disrupts the developer's workflow, reducing productivity.

Solutions:

- The system is fully integrated into the VS Code environment, allowing developers to access it without leaving their coding workspace.
- Queries can be sent directly from VS Code, and responses are displayed within the editor. This seamless integration ensures that developers remain focused on their work, minimizing interruptions and enhancing their overall efficiency.

4. Inconsistent support from existing tools

Problem: Traditional tools like keyword-based search engines fail to handle nuanced queries or provide tailored, actionable solutions.

Solutions:

- The system leverages advanced semantic retrieval techniques through the vector database to locate relevant content based on the meaning of the query rather than just matching keywords.
- The local LLM and vector database ensure an uninterrupted workflow, even in the event of an internet outage.

4.6.1 Local vector storage

The vector storage component is a fundamental part of our system, enabling efficient retrieval of embedded data for the user. Since our primary dataset consists of documentation from the Next.js framework, it inherently contains a combination of both structured and unstructured data:

- **Structured data:** The Next.js documentation is systematically organized. Each documentation page belongs to a specific subject or category, resembling a hierarchical folder structure. This organization provides metadata that aids in filtering and contextual retrieval.
- **Unstructured data:** The text content and code snippets within the documentation represent unstructured data. These elements lack a predefined schema but carry crucial information necessary for coding assistance.

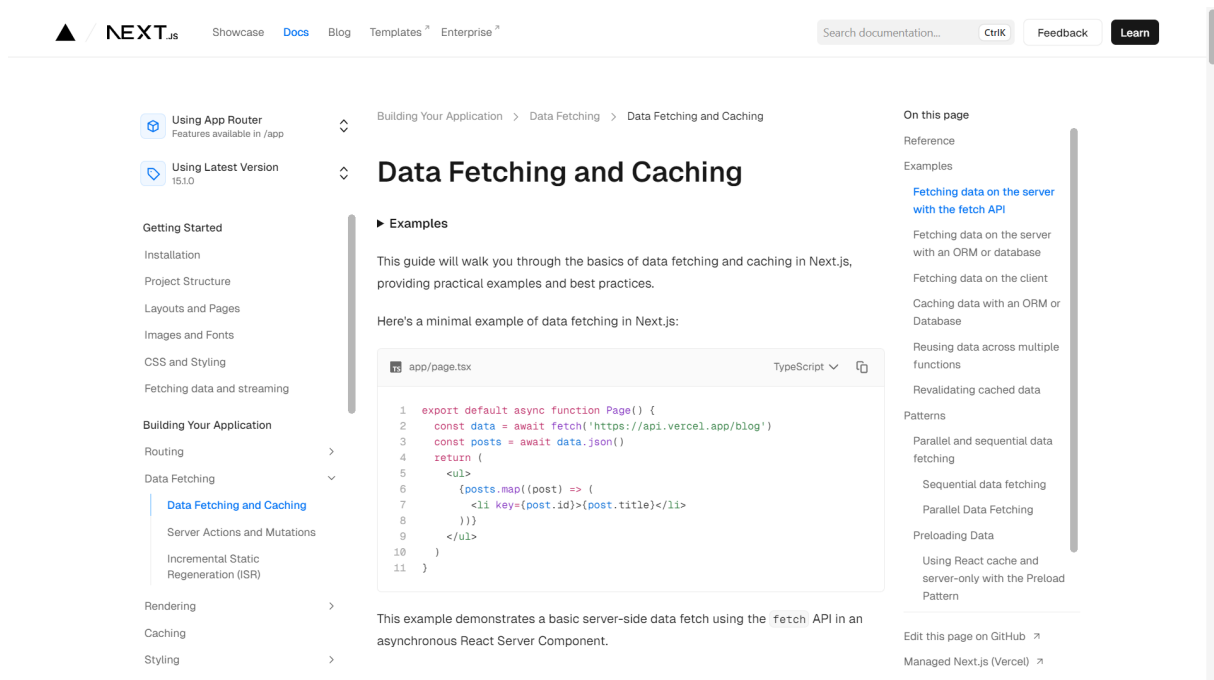


Figure 4.4: Example of a Next.js documentation page

```

---
title: Data Fetching and Caching
nav_title: Data Fetching and Caching
description: Learn best practices for fetching data on the server or
  ↳ client in Next.js.
---

<details>
<summary>Examples</summary>

- [Next.js
  ↳ Commerce](https://vercel.com/templates/next.js/nextjs-commerce)
- [On-Demand ISR](https://on-demand-isr.vercel.app)
- [Next.js
  ↳ Forms](https://github.com/vercel/next.js/tree/canary/examples/next-forms)

</details>

```

This guide will walk you through the basics of data fetching and
 ↳ caching in Next.js, providing practical examples and best
 ↳ practices.

Here's a minimal example of data fetching in Next.js:

```

```tsx filename="app/page.tsx" switcher
export default async function Page() {
 const data = await fetch('https://api.vercel.app/blog')
 const posts = await data.json()
 return (

 {posts.map((post) => (
 <li key={post.id}>{post.title}
))}

)
}
```

```jsx filename="app/page.js" switcher
export default async function Page() {
 const data = await fetch('https://api.vercel.app/blog')
 const posts = await data.json()
 return (

 {posts.map((post) => (
 <li key={post.id}>{post.title}
))}

)
}
```

```

This example demonstrates a basic server-side data fetch using the
 ↳ `fetch` API in an asynchronous React Server Component.

Figure 4.5: Markdown representation of a documentation page

Why Milvus?

Milvus is our chosen vector database for storing embedded data in the system. This decision is based on several compelling reasons:

- **Open source:** Milvus is an open-source database, perfectly aligning with our vision of delivering a free, open-source, distributed code assistance tool for VS Code. Its open nature encourages transparency and community contributions.
- **Support for structured and unstructured data:** The nature of our dataset, comprising both structured metadata (e.g., documentation sections, categories) and unstructured content (e.g., textual descriptions, code snippets), makes Milvus an ideal choice. Structured data benefits from attribute-based filtering, allowing users to narrow their queries to specific documentation sections. Meanwhile, unstructured data, such as free-form text and code snippets, is embedded into high-dimensional vectors for similarity-based retrieval.
- **Comprehensive SDK and model support:** Milvus’s SDK seamlessly integrates with our chosen embedding model, BGE-M3, for generating high-quality embeddings. It also supports the HNSW algorithm for efficient indexing of unstructured data. This compatibility with our preferred methods ensures smooth implementation and reliable performance.
- **Scalability:** Milvus employs LSM-based storage to minimize disk I/O and ensure efficient handling of frequent updates. This scalability is critical for managing dynamic and evolving datasets, such as continuously updated framework documentation.
- **Performance:** Since our system is designed to run entirely on the user’s local machine, Milvus’s ability to leverage both CPU and GPU resources significantly enhances performance. This ensures that computationally intensive tasks, such as vector similarity searches and indexing, are executed efficiently, maintaining a responsive user experience.

| System | Billion-Scale Data | Dynamic Data | GPU | Attribute Filtering | Multi-Vector Query | Distributed System |
|--------------------------------|--------------------|--------------|-----|---------------------|--------------------|--------------------|
| Facebook Faiss [12, 22] | ✓ | ✗ | ✓ | ✗ | ✗ | ✗ |
| Microsoft SPTAG [10] | ✓ | ✗ | ✓ | ✗ | ✗ | ✓ |
| ElasticSearch [13] | ✗ | ✓ | ✗ | ✓ | ✗ | ✓ |
| Jingdong Vearch [26, 27] | ✓ | ✓ | ✓ | ✓ | ✗ | ✓ |
| Alibaba AnalyticDB-V [52] | ✓ | ✓ | ✗ | ✗ | ✗ | ✓ |
| Alibaba PASE (PostgreSQL) [32] | ✗ | ✓ | ✗ | ✓ | ✗ | ✓ |
| Milvus [48] | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

Table 4.3: Vector database comparison

4.6.2 VS Code integration

The interactive user interface is a crucial part of the system, designed to optimize the developer’s experience while working. This component leverages the power of VS Code Extension to provide seamless integration between the development tool and the AI assistant.

Seamless experience in the programming environment: Users can interact directly with the RAG system within their working environment using a VS Code Extension. This is convenient because:

- **No context switching:** By eliminating the need for users to alternate between the code editor and web browser, this feature reduces distractions and boosts efficiency.
- **Direct integration into the project:** The RAG system can quickly access the source code being changed thanks to the VS Code plugin, offering solutions or recommendations without forcing users to copy and paste code into a different interface.

Supportive in IDE environment: The most often used IDE environment for web installers is Visual Studio Code, particularly when using more recent frameworks like Next.js. The RAG system’s VS Code integration enables:

- **Use code context:** From the Next.js structure folder (which includes pages, api, and app) to the file configuration (next.config.js, .env), the system may directly examine the project's coding.
- **Faster debugging and code optimization:** The system may access the coding link to offer more precise answers than the standard web interface information when users need to debug, test logic, or optimize coding.

Security and privacy: When working with source code, especially in sensitive projects, users are often concerned about security issues. A VS Code Extension:

- **Local storage:** Allows the RAG system to operate locally on the user's machine (when using an internal RAG model such as LLaMA), avoiding the need to upload source code to an external server.
- **Limit information leakage:** With extensions, source code and sensitive data do not need to be transferred to a third browser or server, reducing the risk of information disclosure.

This chapter provides a detailed explanation of our implementation strategy based on the outlined approach. The implemented solution consists of three core components: the Database, the RAGGIN core service, and the VSCode extension.

5.1 Database

5.1.1 Data Preparation

We prepared documentation data for Next.js starting from version `v13.0.0` onwards. At the time of writing, the latest version is `v15.3.1`, encompassing a total of 107 supported versions. The data was sourced directly from the official Next.js GitHub repository.

Steps involved in data preparation:

- **Data Acquisition:** A Python script leveraging Git's `sparse-checkout` feature was developed to efficiently download only relevant directories, such as `docs` and `errors`, corresponding to specific version tags. This script automates repository cloning, sparse-checkout configuration, and version-specific checkout.
- **Data Organization:** Retrieved files were systematically organized into directories structured by version. For example, documentation for version `v15.1.0` was organized as follows:

```
./v15.1.0/
  docs/
    01-app/
      index.mdx
    ...
  errors/
    404-get-initial-props.mdx
    amp-bind-jsx-alt.mdx
    ...
```

Upon reviewing the markdown files, we identified several main elements:

- **Metadata:** Essential metadata includes:
 - `title`: Title of the documentation page.
 - `nav_title`: Title displayed in the navigation bar.
 - `description`: Concise summary of the content.
 - `related`: Optional links to related documentation pages.
- **Text Content:** Main documentation content organized into sections and subsections via markdown syntax.

- **Code Snippets:** Each snippet includes:
 - **language:** Programming language (e.g., JavaScript or TypeScript).
 - **file:** Corresponding filename or component name following Next.js conventions.
 - **switcher:** Indicates if an alternative language snippet is available (e.g., JavaScript/TypeScript).
 - **content:** Actual source code.
- **Version History:** A dedicated section describing significant documentation updates across versions, although available in only a limited subset of documents.

After completing data preparation, the quantity of markdown files per version ranged significantly, from 294 files for v13.0.0 to 591 files for v15.3.1. This increase nearly doubled the amount of documentation and documented errors, highlighting substantial growth in the capabilities and complexity of the Next.js framework.

5.1.2 Data Preprocessing

After acquiring and organizing the data, the next critical step was preprocessing it to ensure alignment with our schema design for optimal storage and retrieval. This phase transformed raw markdown files into structured data via a Python-based preprocessing pipeline, leveraging markdown parsing libraries, embedding models, and structured data processing techniques.

Key preprocessing steps

- **Metadata Extraction:** Metadata fields such as `title`, `description`, and `nav_title` were extracted using the `frontmatter` library. Additional metadata, including `related` links when available, were also captured. Unique `entry_ids` were generated for hierarchical referencing within the database schema.

Example code for generating a unique entry ID:

```

1  def generate_entry_id(file_path):
2      relative_path = os.path.relpath(file_path)
3      entry_id = re.sub(r'^[a-zA-Z0-9]', '_', relative_path)
4      return entry_id.lower()
5

```

Listing 5.1: Generating unique entry IDs

- **Markdown parsing:** Markdown files were parsed into components such as headings, paragraphs, lists, and code blocks using the `mistletoe` library. The root token, `Document`, represented the entire markdown file, while specific tokens such as `Heading`, `Paragraph`, and `List` enabled detailed content extraction while maintaining the document's hierarchy.
- **Content extraction:** Each token's child elements were recursively processed to capture plain text, inline code, and hyperlinks. For example, `RawText` tokens represented basic text, `InlineCode` tokens identified inline code snippets, and `Link` tokens extracted hyperlink text and URLs.
- **Section segmentation:** Logical sections were created based on `Heading` tokens in the markdown. Each section included a title, derived from the heading, and grouped content such as paragraphs, lists, or code blocks. This segmentation maintained the hierarchical structure necessary for efficient data retrieval.
- **Code snippet identification:** Code blocks were identified using `CodeFence` tokens and extracted along with relevant metadata, including the programming language, associated filename, and optional switcher flags. This information linked code snippets to their respective sections and allowed for language-specific queries.
- **Embedding preparation:** Text and code content from each section were processed to prepare for embedding generation. Sparse embeddings were used for keyword indexing, while dense embeddings captured semantic meaning, enabling similarity-based queries.

The `parse_markdown_sections` function, implemented using `mistletoe`, played a central role in these steps by iterating over the document tokens to extract structured information. For instance, a document with headings, paragraphs, and code blocks was transformed into hierarchical sections with associated metadata and content.

Example of parsing and organizing code snippets:

```
1 if isinstance(token, CodeFence):
2     code_info = {
3         "code": token.children[0].content if token.children else "",
4         "language": token.language or "text",
5         "filename": filename,
6         "switcher": switcher
7     }
```

Listing 5.2: Code snippet parsing

- **Embedding generation:** Dense and sparse embeddings were generated for both text and code content using a pre-trained embedding model (`BGEM3EmbeddingFunction`). Dense vectors captured semantic meaning, while sparse vectors indexed key terms for efficient retrieval.
- **Storage:** Processed entries were saved in batches as CSV files, with fields such as `entry_id`, `text_content`, and embeddings.

The preprocessing resulted in 107 CSV files, publicly accessible via our dataset NextJs Documentation for RAGGIN, achieving a usability score of 9.41/10 on Kaggle. Dataset insights include:

- **Dataset Size:** From 15.53 MB for version `v13.0.0` up to 49.97 MB for version `v15.3.1`.
- **Entry Count:** Ranges from 396 entries for `v13.0.0` to 1,381 entries for `v15.3.1`.

5.1.3 Database design

Before presenting our schema design, we detail the properties utilized within our fields.

| Properties | Description | Note |
|-------------------------------|---|---|
| <code>name</code> | Name of the field in the collection to create | Data type: String. Mandatory |
| <code>dtype</code> | Data type of the field | Mandatory |
| <code>description</code> | Description of the field | Data type: String. Optional |
| <code>is_primary</code> | Whether to set the field as the primary key field or not | Data type: Boolean (<code>true</code> or <code>false</code>). Mandatory for the primary key field |
| <code>auto_id</code> | Switch to enable or disable automatic ID (primary key) allocation | <code>true</code> or <code>false</code> . Mandatory for primary key field |
| <code>max_length</code> | Maximum byte length for strings allowed to be inserted. | [1, 65,535]. Mandatory for <code>VARCHAR</code> field. Multibyte characters may occupy more than one byte |
| <code>dim</code> | Dimension of the vector | Data type: Integer [1, 32,768]. Mandatory for dense vector field. Omit for sparse vector field |
| <code>is_partition_key</code> | Whether this field is a partition-key field | Data type: Boolean (<code>true</code> or <code>false</code>) |

Table 5.1: Field schema properties(Manage Schema¹)

We developed a schema optimized for organizing and storing documentation data. The schema is implemented within a single collection named `nextjs_docs`.

¹<https://milvus.io/docs/schema.md#Manage-Schema>

| Field | dtype | Len / Dim | Notes / Description |
|--------------------|---------------------|-----------|--|
| entry_id | VARCHAR | 255 | Primary key. auto_id = false. Built by concatenating version + doc name + section name. |
| title | VARCHAR | 255 | Built from documentation name, description, and section name. |
| metadata | JSON | – | JSON blob of title, description, and related links. |
| version | VARCHAR | 20 | Partition key. Indicates documentation version. |
| tag | VARCHAR | 255 | Labels source (documentation vs. documented errors). |
| text_content | VARCHAR | 65 535 | Markdown body. |
| code_content | VARCHAR | 65 535 | JSON-stringified dict of code snippets (language, file name, switcher). |
| sparse_title | SPARSE_FLOAT_VECTOR | – | Sparse embedding of concatenated title + description. <i>Index:</i> SPARSE_INVERTED_INDEX; <i>Metric:</i> Inner Product. |
| dense_text_content | FLOAT_VECTOR | 1024 | Dense embedding of full text. <i>Index:</i> HNSW; <i>Metric:</i> Cosine. |
| dense_code_snippet | FLOAT_VECTOR | 1024 | Avg-pooled vector over all snippet embeddings. <i>Index:</i> HNSW; <i>Metric:</i> Cosine. |

Table 5.2: Schema Structure

When designing the schema, it is crucial to determine how to store code snippets effectively while facilitating the identification of which snippet belongs to which section. Our proposal is to avoid saving code snippets as separate entries in the database in order to reduce latency and computational overhead.

Given this decision, a challenge arises: how should we store embeddings of code snippets, especially since some sections contain none, one, or multiple code snippets? Additionally, for sections with multiple snippets, we must define an efficient storage method.

Figure 5.1 illustrates the distribution of code snippets across sections from version v15.1.4 to v15.3.1. It shows that while some sections have multiple snippets, others have none, reinforcing the need for a flexible storage strategy.

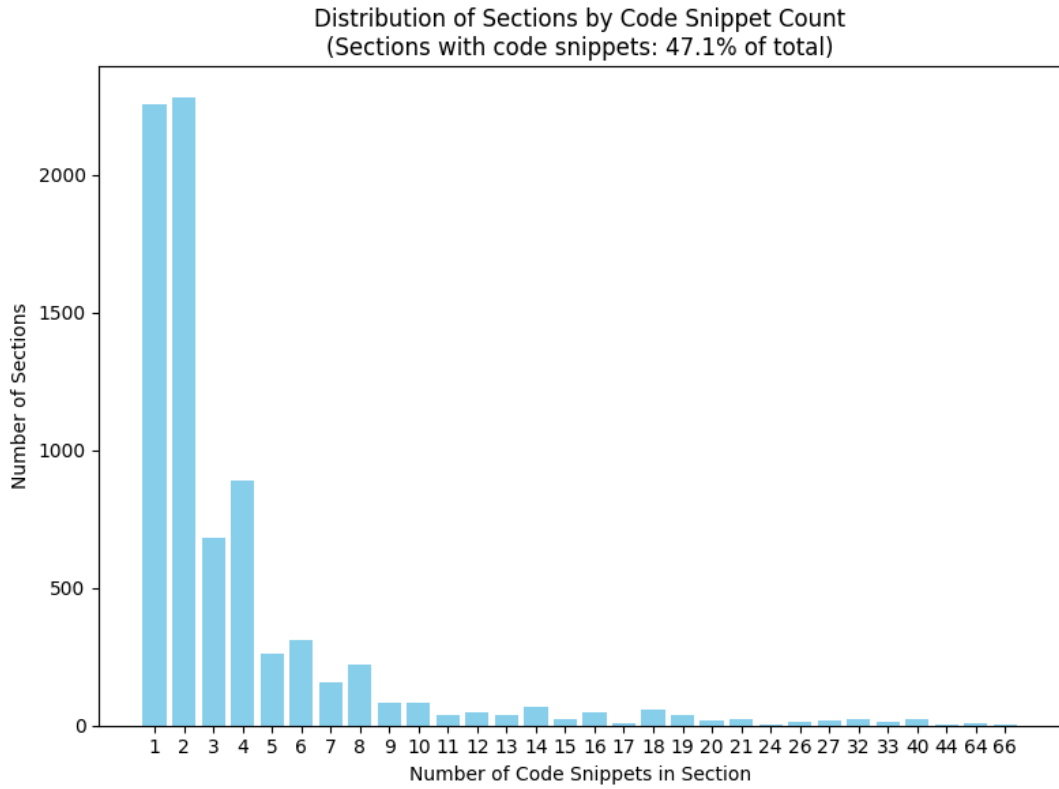


Figure 5.1: Distribution of Code Snippets per Section (v15.1.4 to v15.3.1)

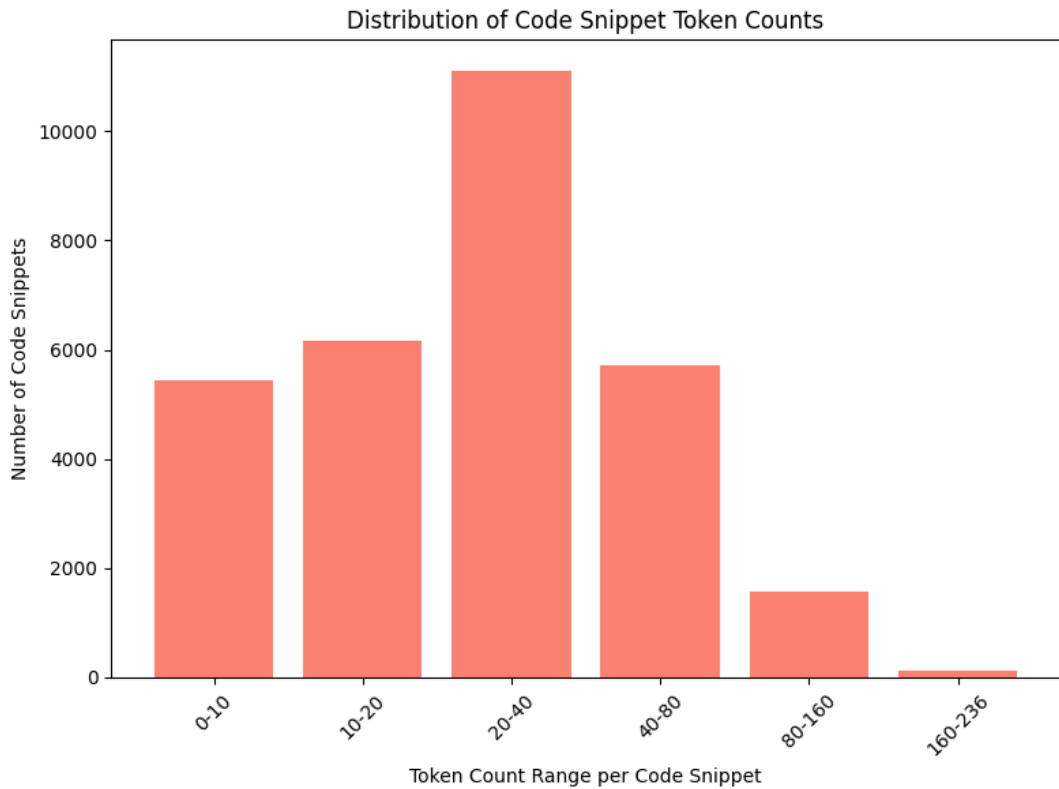


Figure 5.2: Token Count Distribution of Code Snippets (v15.1.4 to v15.3.1)

There are two primary methods for embedding code snippets:

- **Separate Storage:** Each code snippet embedding is stored individually within an array field associated with its section entry.

- **Aggregated Storage:** Embeddings of all code snippets within a section are aggregated into a single vector.

The separate storage method enables precise comparison between a query and individual code snippets. However, it suffers from major drawbacks:

- Additional complexity in indexing entries, as the number of code snippets per section varies.
- Increased database size and computational cost, particularly problematic for large-scale retrieval tasks.

Thus, **aggregated storage** is the preferred option for efficiency.

To unify embeddings into a single vector per section, two aggregation methods are considered:

- **Average Pooling:** Element-wise averaging of all embeddings.
- **Max Pooling:** Element-wise maximum of all embeddings.

To evaluate which aggregation method performs better, we utilize Normalized Discounted Cumulative Gain (NDCG) as the ranking metric.

Figures 5.3 and 5.4 present the NDCG performance of average pooling and max pooling, respectively, evaluated under both dense code search and hybrid search scenarios.

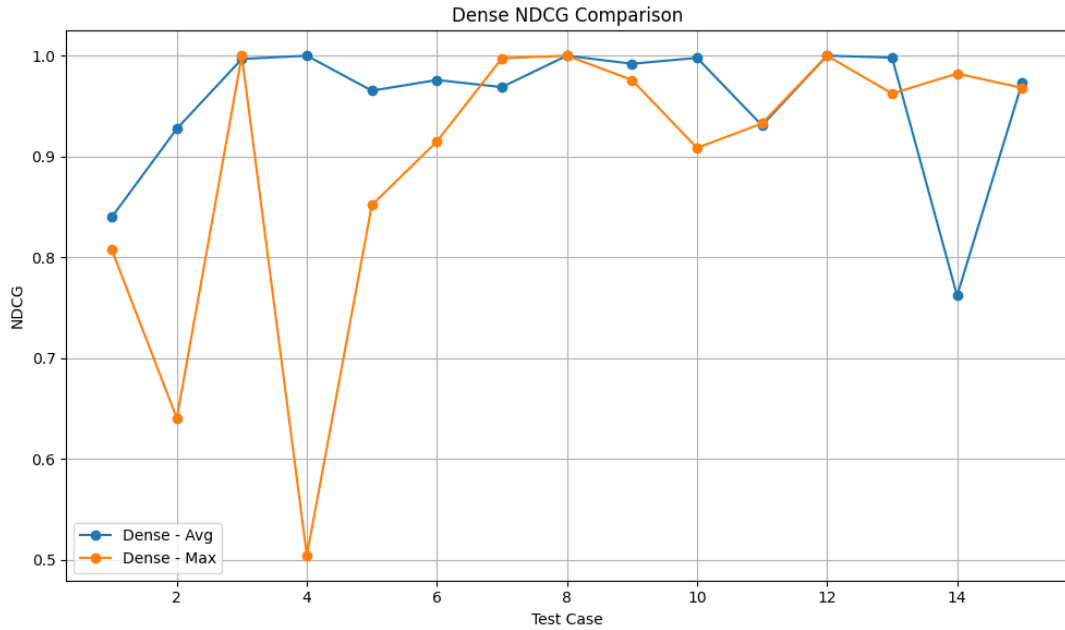


Figure 5.3: NDCG Results with Average Pooling

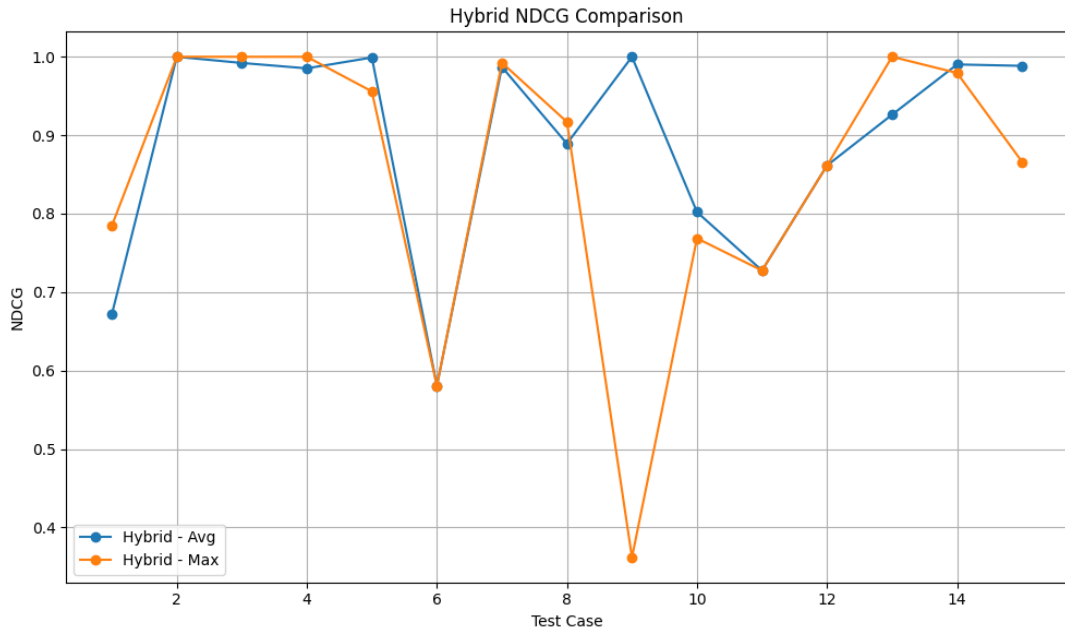


Figure 5.4: NDCG Results with Max Pooling

The evaluation results indicate that average pooling delivers more stable and reliable retrieval performance than max pooling, particularly in maintaining consistent ranking quality.

The retrieved document structure includes a text body containing special placeholder tokens in the form of `code_snippet_{idx}`. During post-processing, each placeholder is replaced by the corresponding code snippet from the `code_content` dictionary.

5.1.4 Database setup

To efficiently handle storage, retrieval, and indexing, we set up our database using Milvus, leveraging Docker Compose for streamlined deployment and configuration. This database setup involved three key services:

- **etcd Service:** We utilized the `etcd` container to manage metadata storage and configuration. It uses the CoreOS `etcd` image version `v3.5.16`. Configuration settings included client URL advertisement, client listening URLs, data directory specification, auto-compaction, and backend quota management to ensure efficient and stable operation.
- **MinIO Service:** The `minio` container was configured for object storage, essential for storing large data objects. The MinIO server uses a specific release version and provides object storage capabilities with secure access via defined credentials. It exposes two ports: one for the main service (port 9000) and another for the console interface (port 9001).
- **Milvus Service (Standalone):** The `milvus-standalone` service, using Milvus version `v2.5.4`, was established for vector indexing and retrieval operations. This container depends on the healthy states of both `etcd` and `minio` services, integrating with them via their respective endpoints. It handles incoming requests and provides monitoring via specified ports.

5.1.5 Database API

The Database API manages documentation data in the Milvus database through seven primary endpoints:

- **List All Supported Versions** [GET `/data/versions`]: Returns a list of all supported Next.js versions with download status.

```

1  [
2    {"version_name": "v13.0.0", "downloaded": true},
3    {"version_name": "v13.0.1", "downloaded": false},

```

```

4     ...
5     {"version_name": "v15.3.1", "downloaded": true}
6 ]

```

Listing 5.3: GET /data/versions output example

- **Retrieve Version Details** [GET /data/versions/{version}]: Fetches metadata (file size, last modified time) for a specific version. Returns 404 if unsupported.

```

1     {
2         "version_name": "v15.3.1",
3         "downloaded": true,
4         "file_size": 49966271,
5         "last_modified": 1745841921.8775475
6     }

```

Listing 5.4: GET /data/versions/version output example

- **List Downloaded Versions** [GET /data/downloaded]: Returns a list of all versions downloaded locally.

```

1     [
2         "v13.0.0",
3         "v15.1.2",
4         "v15.3.1"
5     ]

```

Listing 5.5: GET /data/downloaded output example

- **Retrieve Version Statistics** [GET /data/stats]: Provides statistics including total supported and downloaded versions.

```

1     {
2         "total_supported": 107,
3         "total_downloaded": 3,
4         "downloaded_versions": [
5             "v13.0.0",
6             "v15.1.2",
7             "v15.3.1"
8         ]
9     }

```

Listing 5.6: GET /data/stats output example

- **Retrieve Documentation Version** [POST /version/retrieve]: Downloads a version from Kaggle (if absent) and inserts it into Milvus with optional indexing parameters.

Request parameters:

- **version_name**: Target version (e.g., v15.0.0).
- **m_text** (optional, default = 16): HNSW "M" for text embeddings.
- **ef_text** (optional, default = 200): HNSW "efConstruction" for text embeddings.
- **m_code** (optional, default = 16): HNSW "M" for code embeddings.
- **ef_code** (optional, default = 200): HNSW "efConstruction" for code embeddings.

Processing steps:

1. Check if CSV exists locally.
2. Download CSV from Kaggle if missing.
3. Create Milvus collection if not exists; apply index parameters (M, efConstruction) specific to the version.
 - **M**: Controls graph connectivity in HNSW; higher values improve accuracy but use more memory.
 - **efConstruction**: Controls the candidate pool size during indexing; larger values increase precision at indexing cost.

4. Insert parsed CSV data into Milvus.

- **Delete Documentation Version** [DELETE /version/delete]: Deletes a version from both local storage and Milvus.

Request parameters:

- **version_name**: Target version.
- Optional indexing parameters (**m_text**, **ef_text**, **m_code**, **ef_code**).

Processing steps:

1. Remove local CSV and associated Milvus entries.
2. Download fresh CSV from Kaggle.
3. Recreate Milvus indices using specified or default **M** and **efConstruction** values.
4. Reinsert refreshed data into Milvus.

- **Repair Documentation Version** [POST /version/repair]: Repairs a version by deleting old data and reimporting it fresh.

Request parameters:

- **version_name**: Target version.

Processing steps:

1. Check if CSV exists locally.
2. Remove associated entries from Milvus.
3. Delete the local CSV file.

5.2 RAGGIN core services

The RAGGIN core service provides the API functionalities for hybrid retrieval and generation. It is deployed as a containerized application, built from a custom Dockerfile and exposed through port 8000. The backend interacts directly with the Milvus vector database via a specified internal URI, handling retrieval queries and database operations. It also interfaces with an external Ollama API endpoint to access local LLMs during generation. Pretrained embedding models, such as BGE-M3, are cached within a dedicated volume to reduce startup latency and accelerate embedding generation. The backend depends on the Milvus standalone service for healthy initialization, ensuring database availability before serving requests. Volumes are mounted to persist downloaded documentation files and model checkpoints across restarts, allowing efficient reuse of data. This service acts as the main processing layer connecting the vector storage, retrieval logic, and language model inference.

5.2.1 Retriever

The Retriever service performs hybrid searches by combining sparse lexical matching and dense vector similarity using BGE-M3 embeddings. It leverages Milvus for efficient retrieval and supports weighted multi-modal queries.

Hybrid Search Endpoint [POST /search]

Executes hybrid searches across sparse and dense (text and code) embeddings, returning the top-k most relevant results.

Request parameters:

- **text_query**: Text query for semantic matching.
- **code_query**: Code query for snippet matching.
- **version_name**: Version filter (e.g., v15.0.0).
- **sparse_weight** (optional, default = 1.0): Weight for sparse embeddings.
- **dense_text_weight** (optional, default = 1.0): Weight for dense text embeddings.

- `dense_code_weight` (optional, default = 1.0): Weight for dense code embeddings.
- `top_k` (optional, default = 5): Number of results to return.
- `filter_expr` (optional): Additional filter expression² for Milvus.
- `radius_sparse`, `radius_dense_text`, `radius_dense_code` (optional, default = 0.5): Search radius³ for each modality.
- `range_sparse`, `range_dense_text`, `range_dense_code` (optional, default = 0.5): Search range⁴ for each modality.

Processing steps:

1. Validate input: Ensure at least one modality weight is nonzero; prepare queries and filters.
2. Generate embeddings: Produce sparse and dense vectors using BGE-M3.
3. Search: Query Milvus separately for each modality (sparse, dense text, dense code).
4. Merge results: Raw similarity distances returned by Milvus are mapped to the range $[0, 1]$ to ensure comparability across modalities. The normalization function is defined as:

$$\text{normalized_score} = \frac{\frac{\pi}{2} - \arctan(\text{distance})}{\frac{\pi}{2}}$$

where:

- A distance close to 0 maps to a score close to 1 (indicating high relevance).
- A distance approaching $+\infty$ maps to a score close to 0 (indicating low relevance).
- The output is clamped to $[0, 1]$ to avoid overflow or underflow.

This mapping ensures that smaller distances (closer matches) correspond to higher normalized scores.

After normalization, scores from different modalities (sparse, dense text, dense code) are aggregated into a single combined score using a weighted sum:

$$\text{combined_score} = w_{\text{sparse}} \times s_{\text{sparse}} + w_{\text{dense_text}} \times s_{\text{dense_text}} + w_{\text{dense_code}} \times s_{\text{dense_code}}$$

where:

- $w_{\text{sparse}}, w_{\text{dense_text}}, w_{\text{dense_code}}$ are the user-defined weights for each modality (default 1.0).
- $s_{\text{sparse}}, s_{\text{dense_text}}, s_{\text{dense_code}}$ are the normalized scores from the respective search results.

5. Select top-k: Rank by hybrid scores and return top results.

Example query:

```

1  {
2    "text_query": "Is this command using the canary version?",
3    "code_query": "npx create-next-app@latest",
4    "version_name": "v15.1.2",
5    "sparse_weight": 0.5,
6    "dense_text_weight": 1.0,
7    "dense_code_weight": 0.9,
8    "top_k": 3,
9    "filter_expr": "tag == 'documentation'",
10   "radius_sparse": 0.08,
11   "range_sparse": 1,
12   "radius_dense_text": 0.6,
13   "range_dense_text": 1,
14   "radius_dense_code": 0.6,
15   "range_dense_code": 1
16 }

```

Listing 5.7: POST /search JSON payload example

²Filtered Search - <https://milvus.io/docs/filtered-search.md>

³Range Search - <https://milvus.io/docs/range-search.md>

⁴Range Search - <https://milvus.io/docs/range-search.md>

Example response:

```
1 {
2   "results": [
3     {
4       "title": "02-canary - Upgrade your Next.js Application to canary",
5       "metadata": {},
6       "version": "v15.3.1",
7       "text_content": "...",
8       "code_content": "...",
9       "tag": "documentation",
10      "sparse_distance": 0.115,
11      "dense_text_distance": 0.602,
12      "dense_code_distance": null,
13      "combined_score": 0.745
14    },
15    {
16      "title": "create-next-app - Create Next.js apps",
17      "metadata": {},
18      "version": "v15.3.1",
19      "text_content": "...",
20      "code_content": "...",
21      "sparse_distance": 0.080,
22      "dense_code_distance": 0.901,
23      "combined_score": 0.681
24    },
25    {
26      "title": "12-upgrading - Learn how to upgrade Next.js",
27      "metadata": {},
28      "version": "v15.3.1",
29      "text_content": "...",
30      "code_content": "...",
31      "dense_text_distance": 0.664,
32      "combined_score": 0.626
33    }
34  ]
35 }
```

Listing 5.8: POST /search output example

5.2.2 Generator

Local LLMs setup

In the development of RAG systems, the choice of a robust and secure LLM is critical for achieving high-quality responses and ensuring data privacy. To support our RAG pipeline, we installed and configured Ollama⁵, a platform specifically designed to facilitate the deployment of pre-trained language models on local infrastructure. This section describes the setup and integration of Ollama into our system.

Setup Ollama

The setup process begins by installing the Ollama application, which is compatible with various operating systems including Windows, macOS, and Linux. Once installed, Ollama provides a straightforward interface to download and configure pre-trained models without requiring fine-tuning.

The installation of specific models is conducted via the command prompt or terminal, with available models conveniently listed on <https://ollama.com/search>. The command to install a model is `ollama pull <model_name>`.

Once a model is installed, it is stored locally, ensuring a high level of data privacy and security — an essential feature for applications dealing with sensitive or proprietary information. To test the model, we can use the command `ollama run <model_name>`.

Furthermore, the platform's lightweight design enhances compatibility across a variety of hardware configurations, making it particularly suitable for environments with limited computational resources. This flexibility makes Ollama an optimal choice for deploying local LLMs in diverse use cases.

Integration with the RAG system

To integrate Ollama into the RAG workflow, we use the Ollama API for direct interaction with the locally installed models. The endpoint for the API is:

POST `http://localhost:11434/api/generate`

⁵Ollama - <https://ollama.com/>

The POST request body should be in JSON format, specifying the desired model and the prompt to be processed. An example request body is shown below:

```
1 {
2   "model": "qwen",
3   "prompt": "What is RAG?"
4 }
```

Listing 5.9: Ollama JSON payload example

The API provides a straightforward and efficient method for passing queries and receiving responses from the model.

Generator Endpoint [POST /prompt/generate]

When RAGGIN Generator receives the generate request, it will take the query, model, and file list and process them to RAGGIN Retriever to retrieve relevant documents. Then, it will take the retrieved documents, together with the query and process them to Ollama LLM to generate the final response.

Request parameters:

- **version_name**: Version filter (e.g. v15.3.1).
- **query**: The input query or prompt that serves as the basis for the system to retrieve relevant information and generate a response.
- **model**: The name of the Ollama LLM selected for generating the response.
- **file_list** (Optional): A collection of **FileModel** instances, where each instance includes the following string fields: **fileName**, **fileExtension**, and **fileContent**.
- **additional_options** (Optional): An optional field comprising two subfields: **retriever_options** — the configurable parameters associated with the Hybrid Search Endpoint — and **generator_options** — the set of options controlling the language model's generation behavior. These settings enable users to customize the system's retrieval and generation phases for enhanced flexibility and performance.

Processing steps:

1. Retrieve documents: Use input prompt and version name to retrieve relevance documents.
2. Enhance prompt: Construct an enhanced prompt using the original prompt with the retrieved documents.
3. Generate: Send a POST request to the Ollama API endpoint, then receive and return the generated response.

The generate request's body should be in JSON format, specifying the desired model and the prompt to be processed. An example request body is shown below:

```
1 {
2   "version_name": "v15.1.3",
3   "query": "refactor the files to make it more readable.",
4   "model": "llama2:13b",
5   "additional_options": {
6     "retriever_options": {
7       "top_k": 5
8     },
9     "generator_options": {
10      "temperature": 0.8
11    }
12  },
13  "file_list": [
14    {
15      "file_name": "/my_page.tsx",
16      "file_extension": "tsx",
17      "file_content": "import { File, UserRound, Logout } from \"lucide-react\"; ..."
18    }
19  ]
20 }
```

Listing 5.10: POST /prompt/generate JSON payload example

The response body will contain the generated response and some other relevant information in JSON format.

```
1 {  
2   "model": "llama2:13b",  
3   "response": "Sure! Here are some suggestions for refactoring the code to make it more  
4   readable ..."  
5   ...  
6 }
```

Listing 5.11: POST /prompt/generate output example

5.3 RAGGIN VSCode Extension Development

The RAGGIN (Retrieval-Augmented Generation for Guided Intelligence in Next.js) Visual Studio Code extension is purpose-built to elevate developer productivity within the Next.js ecosystem. RAGGIN operates entirely on the local machine, leveraging Docker and Ollama to remove any reliance on an active internet connection. At its foundation, the extension seamlessly integrates several advanced technologies: local Large Language Models (LLMs) via Ollama for rapid and context-aware response generation; Retrieval-Augmented Generation (RAG) to supplement AI outputs with precise, project-relevant documentation; Docker containers to orchestrate retrieval and computation processes; and the VS Code extension API, which embeds these capabilities natively within the editor. Real-time communication between the VS Code webview and the extension host is achieved through efficient message passing, ensuring a responsive and interactive AI experience.

5.3.1 Installing in VSCode (Private installation)

Install from the VS Code Marketplace

- In your VS Code IDE, go to **Extensions**.
- Search for **Raggin (Self-Hosted)** and select it.
- Install the extension.

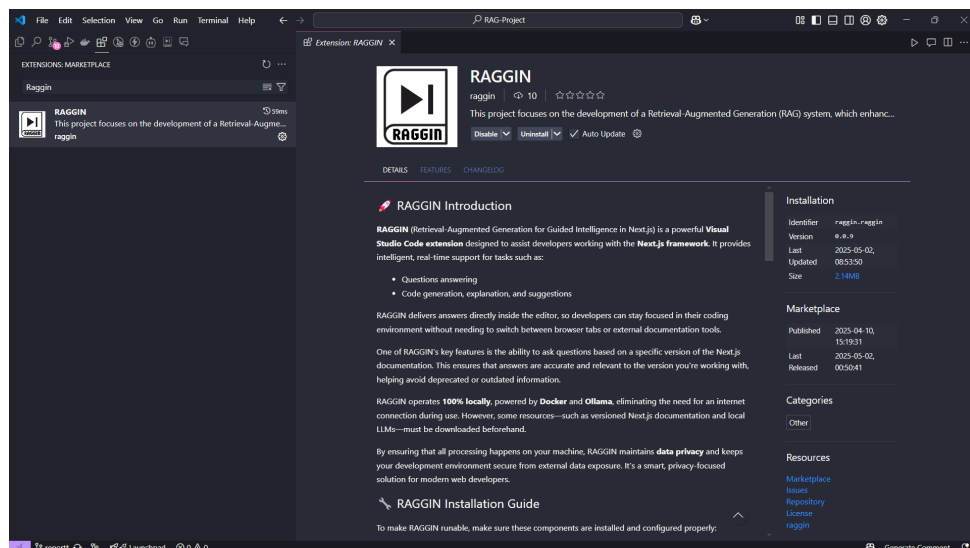


Figure 5.5: Installing Raggin extension in VS Code

You don't need to log in anyway because this is private for user own using.

5.3.2 Icons & labels

Chat page header

The top navigation bar of the IDE plugin displays the selected model (e.g., qwen2.5-coder) and its

corresponding version (e.g., v15.3.0). To the right, users have quick access to Settings, the option to Add a New Chat, and Chat History, streamlining workflow for configuration, testing, and session management.



Figure 5.6: Chat page header

Chat input

This interface includes a file-plus icon on the left, which allows users to upload files from their code base. Once the file is uploaded, it will appear on the left side of the input field, just above the text box, indicating that the file is successfully attached and ready to be referenced or processed within the current chat session. On the far right, there is a send button (shaped like a cleft arrow) used to submit the message. The submit function becomes active after selecting the appropriate Ollama model, specifying the Next.js version, and completing the prompt input.

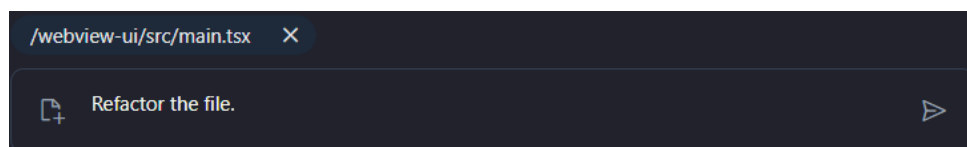


Figure 5.7: Chat input

Added context label

The File Context Label appears prominently beneath the model and version selection bar. It displays the path of the currently active file (e.g., /webview-ui/src/App.tsx), indicating the specific source file that the assistant will use as reference for the current interaction. This context ensures that code-related requests—such as refactoring or debugging—are applied to the correct file within the project structure.

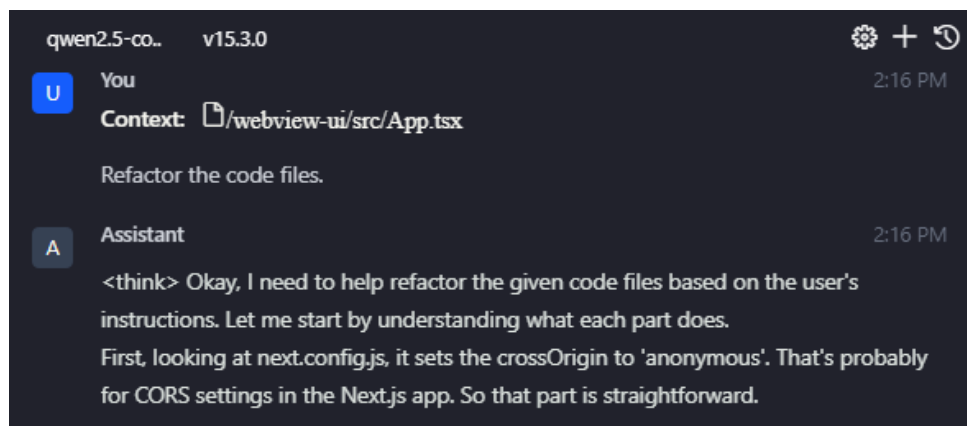


Figure 5.8: Chat input

Chat window

This section of the chat interface displays individual conversation sessions. When a user submits a lengthy question, the interface provides a "Show more / Show less" toggle to optimize readability and minimize vertical scrolling. Responses containing code are rendered using the react-syntax-highlighter library, ensuring clear syntax formatting. Additionally, each code block includes a convenient copy-to-clipboard icon, allowing users to quickly copy the snippet for immediate use in their development workflow.

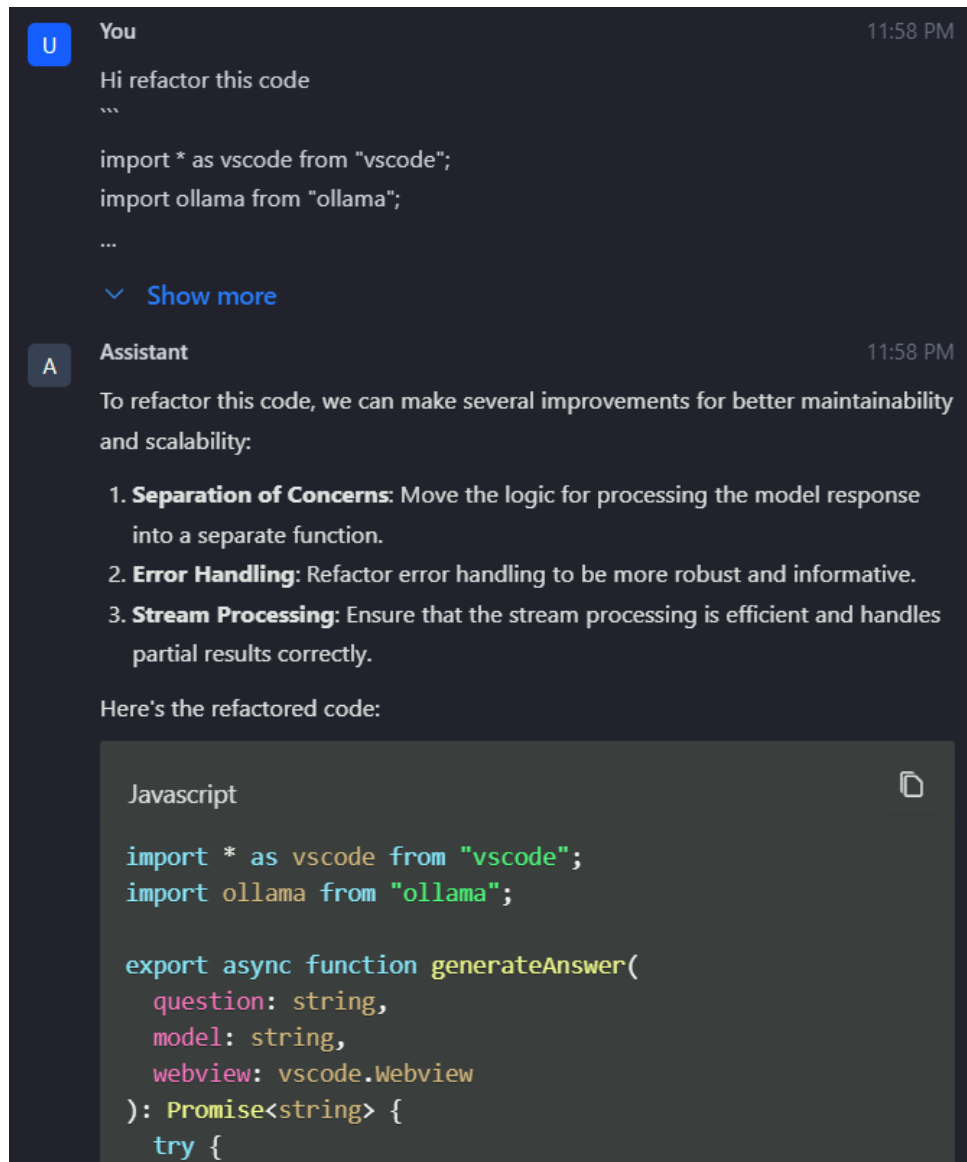


Figure 5.9: Chat input

Chat history header

The Chat History panel serves as a central location for users to access and manage their previous conversation sessions. Positioned at the top of this section is the "Chat History" heading, clearly indicating its purpose. To the right of the heading are two functional icons designed for streamlined interaction. The first icon, represented by a list, allows users to toggle between different view modes or manage existing chat threads more efficiently. Next to it, the clear history icon provides a quick way to delete all saved conversations, helping users maintain a clean workspace.



Figure 5.10: Chat history header

Chat history window

Each chat entry in the chat history panel includes a three-dot settings menu located in the top-right corner of the chat card. This contextual menu provides users with the ability to rename the conversation—allowing for more meaningful and descriptive titles—or to delete the chat entirely when it is no longer needed. These options help users efficiently manage their session history and maintain an organized workspace.

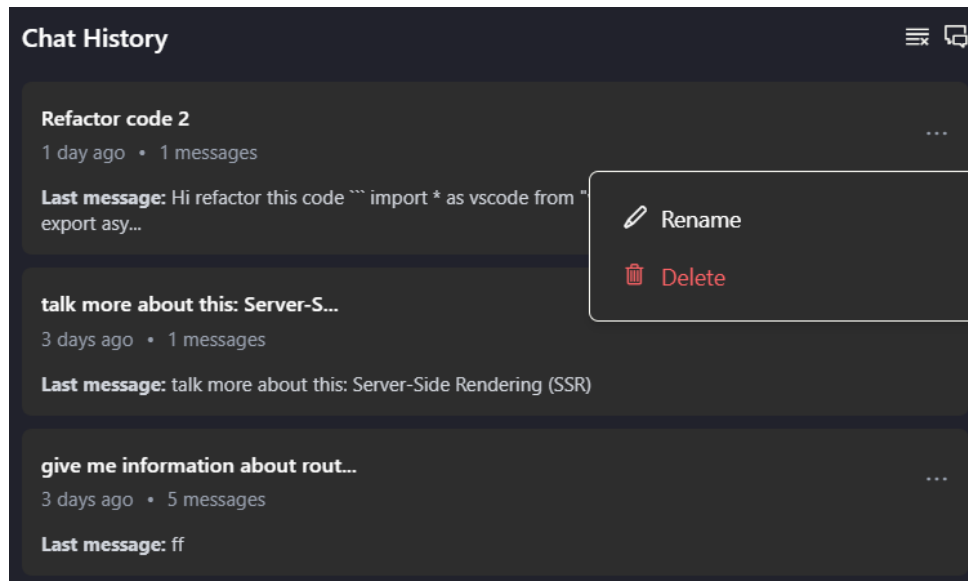


Figure 5.11: Chat history window

Advance settings page

The interface displayed is the Advanced Settings panel, designed to fine-tune parameters for both retrieval and generation in RAG workflow. At the top, there is a tabbed header with two options: Retriever options (currently selected) & Generator options. Each section includes an Instructions link that likely directs users to documentation or usage guidance. At the bottom, users can choose to Cancel changes or Save their configuration using clearly labeled action buttons.

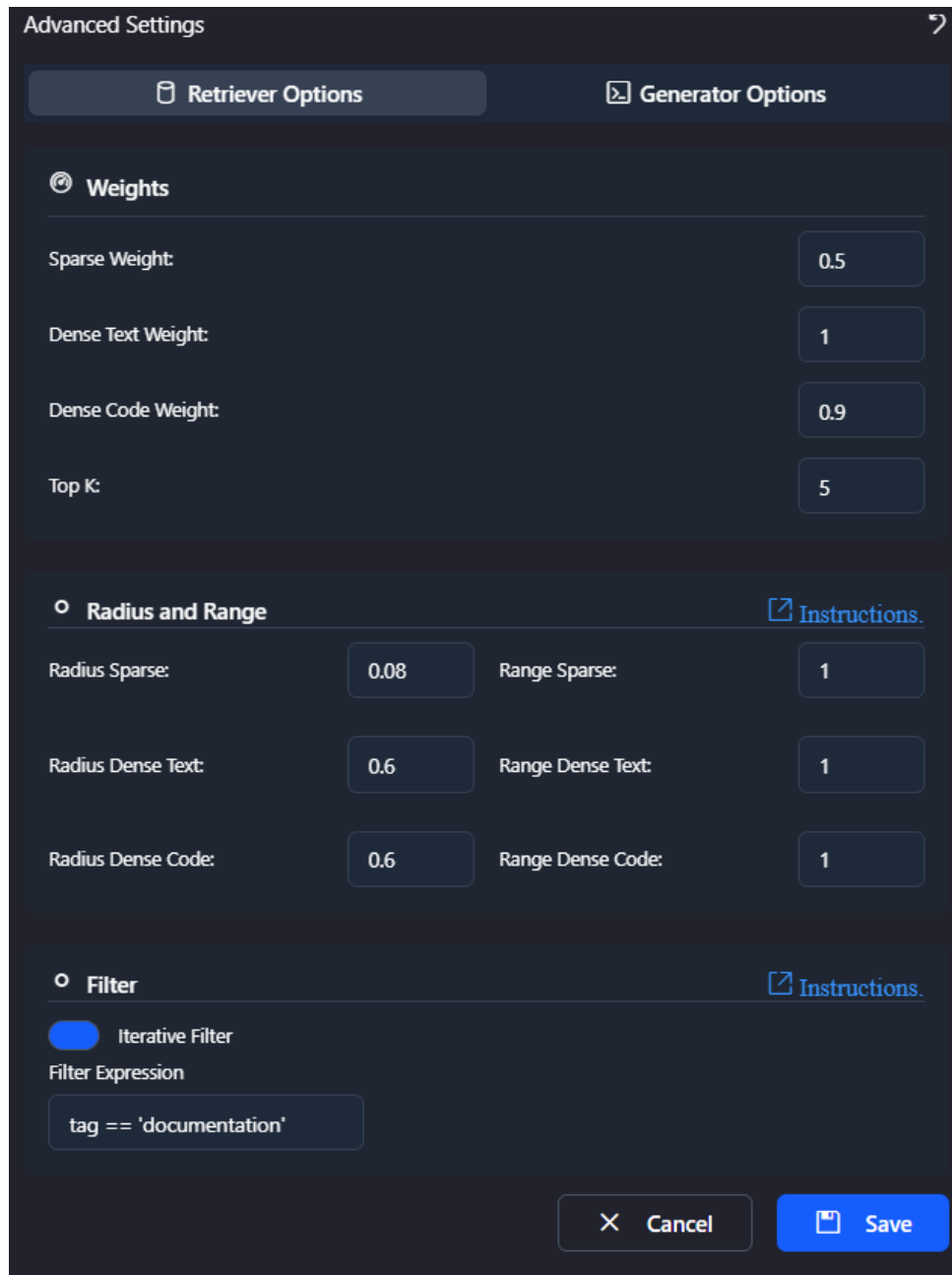


Figure 5.12: Advance settings page

5.3.3 Primary function

RAGGIN is a lightweight, privacy-preserving AI assistant extension built for Visual Studio Code, specifically tailored to support Next.js developers across a wide range of versions. The extension integrates Retrieval-Augmented Generation (RAG) techniques to deliver context-aware, intelligent responses directly within the development environment.

Key features include:

- **Real-time Querying within VSCode:** Developers can interact with the AI assistant directly inside the extension panel. The primary focus is on Next.js-related queries, with dynamic control over the Next.js version (from v13.0.0 to v15.3.1) to ensure framework-specific accuracy. Additionally, users can switch between Ollama-hosted LLM models, offering flexibility in model selection based on task complexity or system capabilities.
- **Local Contextual Intelligence:** Users can load a specific code file from their local project to provide the AI with deeper context. All operations occur within the local development environment, ensuring data privacy and security—no external server calls or remote data exposure

- **Persistent Chat History:** All user interactions are stored locally using VSCode’s globalState API, enabling developers to retain conversation history across sessions. The dedicated history view allows quick reference and reuse of past discussions without re-prompting.
- **Advanced Configuration Panel:** For power users and researchers, the Advanced Settings page provides control over key RAG parameters:
 - Retrieval method customization (e.g., top-k, similarity threshold)
 - Generator configuration (e.g., temperature, max tokens) These tunable settings support experimentation and fine-tuning for optimal results.

5.3.4 Performance

Raggin stands out as one of the most lightweight AI-assist extensions on the VSCode Marketplace, with a package size under 2.5MB. Optimized for speed, memory efficiency, and modular development, it consumes minimal system resources—using CPU and memory at levels comparable to or lower than typical VSCode extensions. The extension achieves this efficiency by offloading most computational work to the underlying RAG model via Ollama, ensuring your editor remains responsive even during complex queries. Response quality and speed scale with your hardware capabilities—systems that can run larger LLMs will experience enhanced performance with faster, more accurate answers.

5.3.5 Known Limitations and Future Roadmap

Known Limitations: Despite its current utility, RAGGIN still has areas for growth. Currently, the extension supports contextual referencing for **one file at a time**, which may restrict its effectiveness in multi-module projects. Additionally, responses are only rendered **after full generation**, lacking token-by-token streaming, which may result in perceived delays for long outputs.

Planned Enhancements: The development roadmap for RAGGIN includes several key improvements: implementing **streaming responses** for real-time feedback, upgrading the **user interface** for smoother interaction and better usability, enabling **multi-file context handling** to support complex codebases, and enhancing **prompt management and template customization** for more structured queries.

6.1 Evaluation metrics

6.1.1 Retriever Metrics

Recall@K: Recall@K quantifies the proportion of relevant documents successfully retrieved among the top K results for a given query.

$$\text{Recall@K} = \frac{|\text{Relevant documents} \cap \text{Top-}K \text{ retrieved documents}|}{|\text{Relevant documents}|}$$

Where:

- $|\text{Relevant documents}|$ denotes the total number of documents relevant to the query.
- $|\text{Relevant documents} \cap \text{Top-}K \text{ retrieved documents}|$ is the count of relevant documents found in the top K results.

Example:

- Scenario: A query returns the top $K = 5$ documents. Among them, 3 are relevant, while the total number of relevant documents is 4.
- Calculation:

$$\text{Recall@5} = \frac{3}{4} = 0.75 \quad (75\%)$$

Precision@K: Precision@K measures the accuracy of the top K retrieved documents by evaluating how many of them are truly relevant.

$$\text{Precision@K} = \frac{|\text{Relevant documents} \cap \text{Top-}K \text{ retrieved documents}|}{|\text{Top-}K \text{ retrieved documents}|}$$

Where:

- $|\text{Top-}K \text{ retrieved documents}|$ is the number of documents retrieved within the top K results.

Example:

- Scenario: A query retrieves the top $K = 5$ documents, with 3 being relevant. The total number of relevant documents is 4.
- Calculation:

$$\text{Precision@5} = \frac{3}{5} = 0.6 \quad (60\%)$$

Latency: Latency represents the response time taken to complete a retrieval operation.

$$\text{Latency (ms)} = \text{Time}_{\text{returned}} - \text{Time}_{\text{submitted}}$$

Example:

- Scenario: The query is submitted at 11:00:00.000 and the result is received at 11:00:00.340.
- Calculation:

$$\text{Latency} = 340 \text{ ms} - 0 \text{ ms} = 340 \text{ ms}$$

6.1.2 Generator Metrics

ROUGE Score: ROUGE evaluates the lexical overlap between generated content and a set of reference texts. ROUGE-N specifically computes n -gram overlaps:

$$\text{ROUGE-N} = \frac{\sum_{\text{ref} \in \text{References}} \sum_{\text{ngram} \in \text{Generated}} \text{Count}(\text{ngram})}{\sum_{\text{ref} \in \text{References}} \sum_{\text{ngram} \in \text{ref}} \text{Count}(\text{ngram})}$$

Example:

- Scenario: A reference document has 10 bigrams, and the generated output contains 8 matching bigrams.
- Calculation:

$$\text{ROUGE-2} = \frac{8}{10} = 0.8$$

Relevance: Relevance measures how well the generated outputs align with the intent of user queries, typically rated manually and averaged across queries.

$$\text{Relevance} = \frac{1}{N} \sum_{i=1}^N \text{RelevanceScore}_i$$

Example:

- Scenario: Five queries yield relevance scores of 0.9, 0.8, 0.7, 1.0, and 0.6.
- Calculation:

$$\text{Relevance} = \frac{0.9 + 0.8 + 0.7 + 1.0 + 0.6}{5} = 0.8$$

Latency: Latency in the generation phase denotes the time elapsed between the completion of retrieval and the generation of the final output.

$$\text{Latency (ms)} = \text{Time}_{\text{generated}} - \text{Time}_{\text{retrieved}}$$

Example:

- Scenario: The output is generated at 12:00:00.210 after retrieval completes at 12:00:00.000.
- Calculation:

$$\text{Latency} = 210 \text{ ms} - 0 \text{ ms} = 210 \text{ ms}$$

6.2 Evaluation methodology

To evaluate the performance of both the retriever and the generator, we employ a curated dataset containing queries with predefined relevant documents. The evaluation covers two types:

- **Simple queries:** Direct questions that require retrieving clearly defined information from the documentation.
- **Complex queries:** Queries involving multiple components, conditions, or requiring deeper semantic understanding.

6.3 Evaluation summary

6.3.1 Retriever Performance

Table B.1 and Table B.2 present the retriever performance and generator latency for simple and complex queries, respectively.

Results show that the retriever achieved high **Recall@K** in simple queries, with a peak of **0.81** at **K=5**, demonstrating strong coverage of relevant documents. However, performance declined in complex queries due to their nuanced requirements, with **Recall@K** dropping to **0.24** at **K=5**. **Precision@K** also showed a similar pattern, peaking at **0.57** in simple cases but falling below **0.3** for complex ones.

Latency remained reasonable for simple queries (approximately **1.2 seconds**), while complex queries understandably took longer due to the increased search depth and token length, averaging around **3.4 seconds**.

6.3.2 Generator Performance

The generator ROUGE scores across various models are illustrated in Figure B.1, Figure B.2 and Figure B.3, corresponding to ROUGE-1, ROUGE-2 and ROUGE-3 metrics, respectively. Additionally, model-wise relevance evaluations are presented in Figure B.5 and Figure B.6, capturing the perceived quality of generated outputs across different model sizes. Notably:

- **Codestral-22B**, **LLaMA2-13B**, and **Qwen2.5-14B** consistently achieved top scores, producing highly relevant and fluent outputs across diverse test cases.
- **Mistral-7B**, **CodeLLaMA-7B**, and **Qwen2.5-3B** also demonstrated balanced performance, with strong semantic accuracy and reliable response generation.
- Smaller models like **Qwen-1.8B** and **CodeGemma-2B** showed limitations in deeper contextual understanding, reflected in lower performance in complex reasoning tasks.

Overall, generation latency remained within practical limits—averaging approximately **6 seconds** for simple prompts and **12 seconds** for complex queries—though these values varied depending on the size and architecture of the underlying language model.

6.4 Testing

6.4.1 Unit testing

Validate individual functions and components of the extension.

6.4.2 Integration testing

Ensure proper interaction between extension components and the VSCode environment.

Conclusion

This chapter summarizes the accomplishments of the project over two semesters of research, design, and implementation. It also outlines the current limitations and potential directions for future development of the extension.

7.1 Achieved Results

We have developed an intelligent code assistance system tailored for web development, leveraging RAG to reduce hallucination and support integration of up-to-date information. The system is implemented as a Visual Studio Code extension named RAGGIN, Retrieval-Augmented Generation for Guided Intelligence in Next.js.

The project's source code is publicly available on GitHub under the MIT license¹, ensuring full transparency. The backend services² have been containerized and published on Docker Hub, while the extension frontend³ is released on the Visual Studio Code Marketplace. Additionally, the processed Next.js documentation dataset used for retrieval has been made available on Kaggle⁴.

We have delivered a system that assists developers in integrating AI into their workflow, offering an open-source solution that grants the freedom to choose their preferred LLMs according to their system requirements. The system provides both deep configurability and flexibility while maintaining seamless integration within their familiar development environment.

7.2 Limitations

At present, the extension focuses exclusively on the Next.js framework, without support for the broader ecosystem of related technologies commonly used alongside it. While we have manually curated the documentation dataset, defined chunking strategies, and designed embedding pipelines specifically for Next.js, the system lacks the ability to automatically identify and process documentation from other frameworks. As a result, the extension cannot yet be scaled automatically to support additional technologies.

Currently, RAGGIN only offers configuration flexibility for the generator model (i.e., the LLM used for response generation). The embedding model for documentation remains fixed, with embeddings precomputed and bundled with the system. Moreover, the extension does not yet support local GPU-based embedding generation for user queries due to the complexities involved in GPU Docker setup and the additional size such support would impose on the container image.

In terms of retrieval performance, our core service exhibits low precision and recall at small Top- K values: Precision@1 is approximately 0.267, and Recall@1 is 0.053. Performance improves only at larger K , indicating the retrieval system is currently not optimized for extracting highly relevant results with minimal context. This limitation negatively impacts downstream LLM performance, as more irrelevant documents are included in the prompt context.

¹The MIT License — <https://opensource.org/license/mit>

²RAGGIN core service — <https://hub.docker.com/repository/docker/melukootto/raggin/general>

³RAGGIN Extension — <https://marketplace.visualstudio.com/items/?itemName=raggin.raggin>

⁴Next.js documentation dataset — <https://www.kaggle.com/datasets/jiyujizai/nextjs-documentation-for-raggin>

The full system image occupies roughly 20GB on Docker (17.3GB for the core backend service and 2.7GB for the Milvus database), excluding the language model and Ollama runtime, which challenges our original lightweight deployment goals.

Finally, it is worth noting that RAGGIN is positioned as an open-source alternative to proprietary services like GitHub Copilot⁵, which benefits from commercial-scale infrastructure and full integration with the Visual Studio Code ecosystem through its parent company, Microsoft.

7.3 Future development directions

To address the current limitations, we propose several concrete development directions aimed at improving scalability, configurability, performance, and portability of RAGGIN.

Expanding framework support To broaden beyond Next.js, future work will focus on automating the identification and acquisition of documentation from other technologies, such as React⁶, Vue⁷, or TailwindCSS⁸. This may involve using sitemap parsing, structured data from `schema.org`⁹, or scraping via documentation RSS feeds. A modular preprocessing pipeline will also be developed to support customizable chunking and embedding per framework.

Custom embedding model configuration Support for configurable embedding models will be introduced, allowing users to select their preferred models (e.g., BGE-M3, E5, or InstructorXL¹⁰) during setup. Additionally, GPU-based embedding generation for user queries will be optionally enabled through a separate, GPU-enabled Docker image to prevent bloating the default container.

Improving retrieval accuracy at low Top-K Include fine-tuning the embedding model on task-specific data, introducing learning-to-rank mechanisms for hybrid scoring, and employing lightweight rerankers such as MiniLM¹¹ or RankT5¹² to refine the top retrieved entries. These methods aim to increase the relevance of results with minimal context.

Reducing system size To meet lightweight deployment goals, we plan to modularize services, separating the backend API, and embedding module into independently deployable containers. Further size reduction can be achieved through multi-stage Docker builds, minimal base images (e.g., Alpine¹³).

Ecosystem integration and tooling To enhance usability and ecosystem compatibility, future development will target integration with the Language Server Protocol (LSP)¹⁴, enabling support beyond Visual Studio Code (e.g., WebStorm¹⁵, Neovim¹⁶).

These future directions aim to evolve RAGGIN into a robust, extensible, and developer-friendly tool that remains open, transparent, and adaptable to diverse workflows.

⁵GitHub Copilot — <https://github.com/features/copilot>

⁶React — <https://reactjs.org>

⁷Vue — <https://vuejs.org>

⁸Tailwind CSS — <https://tailwindcss.com>

⁹`schema.org` — <https://schema.org>

¹⁰InstructorXL — <https://huggingface.co/hkunlp/instructor-xl>

¹¹MiniLM — <https://huggingface.co/nreimers/MiniLM-L6-H384-uncased>

¹²RankT5 — <https://github.com/nyu-dl/dl4marco-2022>

¹³Alpine Linux — <https://alpinelinux.org>

¹⁴Language Server Protocol (LSP) — <https://microsoft.github.io/language-server-protocol>

¹⁵WebStorm — <https://www.jetbrains.com/webstorm>

¹⁶Neovim — <https://neovim.io>

Bibliography

- [1] A. Asai et al. “SELF-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint* arXiv:2310.11511v1 (2023). Available at <https://arxiv.org/abs/2310.11511>.
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer Normalization”. In: *stat* 1050 (2016), p. 21.
- [3] Jinze Bai et al. “Qwen Technical Report”. In: *arXiv preprint* arXiv:2309.16609 (2023). Available at <https://arxiv.org/abs/2309.16609>.
- [4] Erik Bernhardsson. *Approximate Nearest Neighbors Oh Yeah*. Online. Available at <https://github.com/spotify/annoy>.
- [5] D. Brown. *Rank-BM25: A Collection of BM25 Algorithms in Python*. Online. Available at <https://doi.org/10.5281/zenodo.4520057>. 2020.
- [6] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: <https://arxiv.org/abs/2005.14165>.
- [7] Marcel Bruch, Martin Monperrus, and Mira Mezini. “Learning from Examples to Improve Code Completion Systems”. In: *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and ACM Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Amsterdam, Netherlands, 2009.
- [8] Jianlv Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. arXiv: 2402.03216 [cs.CL]. URL: <https://arxiv.org/abs/2402.03216>.
- [9] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: <https://arxiv.org/abs/2107.03374>.
- [10] Qi Chen et al. *SPTAG: A Library for Fast Approximate Nearest Neighbor Search*. Available at <https://github.com/Microsoft/SPTAG>. 2018.
- [11] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Available at <https://arxiv.org/abs/1810.04805>. 2019. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- [12] Matthijs Douze et al. “The Faiss library”. In: (2024). arXiv: 2401.08281 [cs.LG].
- [13] *Elasticsearch*. Online. Available at <https://github.com/elastic/elasticsearch>.
- [14] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint* arXiv:2312.10997 (2023). Available at <https://arxiv.org/abs/2312.10997>.
- [15] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [16] Pengfei He et al. “Exploring Demonstration Retrievers in RAG for Coding Tasks: Yeas and Nays!” In: *arXiv preprint* arXiv:2410.09662v1 (2024). Available at <https://arxiv.org/abs/2410.09662>.
- [17] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-term Memory”. In: *Neural computation* 9 (Dec. 1997), pp. 1735–80. DOI: 10.1162/neco.1997.9.8.1735.

- [18] Marko Hostnik and Marko Robnik-Sikonja. “Retrieval-Augmented Code Completion for Local Projects Using Large Language Models”. In: *arXiv preprint* arXiv:2408.05026v1 (2024). Available at <https://arxiv.org/abs/2408.05026>.
- [19] Binyuan Hui et al. “Qwen2.5-Coder Technical Report”. In: *arXiv preprint* arXiv:2409.12186 (2024). Available at <https://arxiv.org/abs/2409.12186>.
- [20] Gautier Izacard et al. “Unsupervised Dense Information Retrieval with Contrastive Learning”. In: *Transactions on Machine Learning Research* (2022). ISSN: 2835-8856. URL: <https://openreview.net/forum?id=jKN1pXi7b0>.
- [21] O. Jafari et al. “A Survey on Locality Sensitive Hashing Algorithms and Their Applications”. In: *arXiv preprint* arXiv:2102.08942 (2021). Available at <https://arxiv.org/abs/2102.08942>.
- [22] Jeff Johnson, Matthijs Douze, and Hervé Jégou. “Billion-scale Similarity Search with GPUs”. In: *arXiv preprint* arXiv:1702.08734 (2017). Available at <https://arxiv.org/abs/1702.08734>.
- [23] Aadit Kshirsagar. “Enhancing RAG Performance Through Chunking and Text Splitting Techniques”. In: *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* 10 (Sept. 2024), pp. 151–158. DOI: 10.32628/CSEIT2410593.
- [24] Mandar Kulkarni et al. “Reinforcement Learning for Optimizing RAG for Domain Chatbots”. In: *Proceedings of the AAAI 2024 Workshop on Synergy of Reinforcement Learning and Large Language Models*. 2024.
- [25] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *arXiv preprint* arXiv:2005.11401 (2020).
- [26] Jie Li et al. “The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform”. In: *arXiv preprint* arXiv:1908.07389 (2019). Available at <https://arxiv.org/abs/1908.07389>.
- [27] Jie Li et al. “The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform”. In: *arXiv preprint* arXiv:1908.07389 (2019). Available at <https://arxiv.org/abs/1908.07389>.
- [28] Xinze Li et al. “Structure-Aware Language Model Pretraining Improves Dense Retrieval on Structured Data”. In: *arXiv preprint* arXiv:2305.19912 (2023). Available at <https://arxiv.org/abs/2305.19912>.
- [29] Y. A. Malkov and D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.
- [30] Tomas Mikolov et al. “Efficient Estimation of Word Representations in Vector Space”. In: *arXiv preprint arXiv:1301.3781* (2013).
- [31] Bhaskar Mitra and Nick Craswell. “Neural Models for Information Retrieval”. In: *arXiv preprint* arXiv:1705.01509v1 (May 2017). Available at <https://arxiv.org/abs/1705.01509>.
- [32] *PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension*. Online. Available at <https://www.researchgate.net/publication/341744935>.
- [33] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. “GloVe: Global Vectors for Word Representation”. In: *Conference on Empirical Methods in Natural Language Processing*. 2014. URL: <https://api.semanticscholar.org/CorpusID:1957433>.
- [34] Alec Radford and Karthik Narasimhan. “Improving Language Understanding by Generative Pre-Training”. In: 2018. URL: <https://api.semanticscholar.org/CorpusID:49313245>.
- [35] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. Available at https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. OpenAI, 2019.
- [36] Colin Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Available at 1910.10683. 2023. arXiv: 1910.10683 [cs.LG]. URL: <https://arxiv.org/abs/1910.10683>.
- [37] O. Ram et al. “In-Context Retrieval-Augmented Language Models”. In: *arXiv preprint* arXiv:2302.00083v3 (2023). Available at <https://arxiv.org/abs/2302.00083>.

- [38] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Nov. 2019. URL: <https://arxiv.org/abs/1908.10084>.
- [39] Stephen E. Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Found. Trends Inf. Retr.* 3 (2009), pp. 333–389. URL: <https://api.semanticscholar.org/CorpusID:207178704>.
- [40] Baptiste Rozière et al. “Code Llama: Open Foundation Models for Code”. In: *arXiv preprint arXiv:2308.12950* (2023). Available at <https://arxiv.org/abs/2308.12950>.
- [41] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986). Available at <https://doi.org/10.1038/323533a0>, pp. 533–536.
- [42] Saksham Sachdev et al. “Retrieval on Source Code: A Neural Code Search”. In: *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. Philadelphia, PA, USA, 2018.
- [43] Pranab Sahoo et al. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. 2024. arXiv: 2402.07927 [cs.AI]. URL: <https://arxiv.org/abs/2402.07927>.
- [44] Alexey Svyatkovskiy et al. “IntelliCode Compose: Code Generation using Transformer and Retrieval-based Models”. In: *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*. 2020.
- [45] Hanzhuo Tan et al. “Prompt-based Code Completion via Multi-Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2405.07530v1* (2024). Available at <https://arxiv.org/abs/2405.07530>.
- [46] Gemini Team et al. *Gemini: A Family of Highly Capable Multimodal Models*. 2024. arXiv: 2312.11805 [cs.CL]. URL: <https://arxiv.org/abs/2312.11805>.
- [47] Ashish Vaswani et al. “Attention Is All You Need”. In: *arXiv preprint arXiv:1706.03762* (2017). Available at <https://arxiv.org/pdf/1706.03762>.
- [48] Jianguo Wang et al. “Milvus: A Purpose-Built Vector Data Management System”. In: *Proceedings of the 2021 International Conference on Management of Data (SIGMOD ’21)*. Virtual Event, China, 2021, pp. 2624–2638. DOI: 10.1145/3448016.3457550.
- [49] Liang Wang et al. *Improving Text Embeddings with Large Language Models*. 2024. arXiv: 2401.00368 [cs.CL]. URL: <https://arxiv.org/abs/2401.00368>.
- [50] Liang Wang et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. 2024. arXiv: 2212.03533 [cs.CL]. URL: <https://arxiv.org/abs/2212.03533>.
- [51] X. Wang et al. “KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases”. In: *arXiv preprint arXiv:2308.11761v1* (2023). Available at <https://arxiv.org/abs/2308.11761>.
- [52] Chuangxian Wei et al. “AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 3152–3165. DOI: 10.14778/3415478.3415541.
- [53] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [54] Di Wu et al. “Repoformer: Selective Retrieval for Repository-Level Code Completion”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Vienna, Austria: PMLR, 2024.
- [55] Chanwoong Yoon et al. “COMPACT: Compressing Retrieved Documents Actively for Question Answering”. In: *Korea University and AIGEN Sciences* (2024). Authors: Chanwoong Yoon, Taewho Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang.
- [56] Fengji Zhang et al. “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation”. In: *arXiv preprint arXiv:2303.12570* (2023). Available at <https://arxiv.org/abs/2303.12570>.

- [57] Xinyu Zhang et al. “MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages”. In: *Transactions of the Association for Computational Linguistics* 11 (Sept. 2023), pp. 1114–1131. ISSN: 2307-387X. DOI: 10.1162/tac1_a_00595. eprint: https://direct.mit.edu/tac1/article-pdf/doi/10.1162/tac1_a_00595/2157340/tac1_a_00595.pdf. URL: https://doi.org/10.1162/tac1%5C_a%5C_00595.
- [58] Xinyu Zhang et al. “Toward Best Practices for Training Multilingual Dense Retrieval Models”. In: *ACM Trans. Inf. Syst.* 42.2 (Sept. 2023). ISSN: 1046-8188. DOI: 10.1145/3613447. URL: <https://doi.org/10.1145/3613447>.
- [59] Ziyao Zhang et al. “LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation”. In: *arXiv preprint* arXiv:2409.20550v1 (2024). Available at <https://arxiv.org/abs/2409.20550>.

Appendices

Interface of the RAGGIN extension

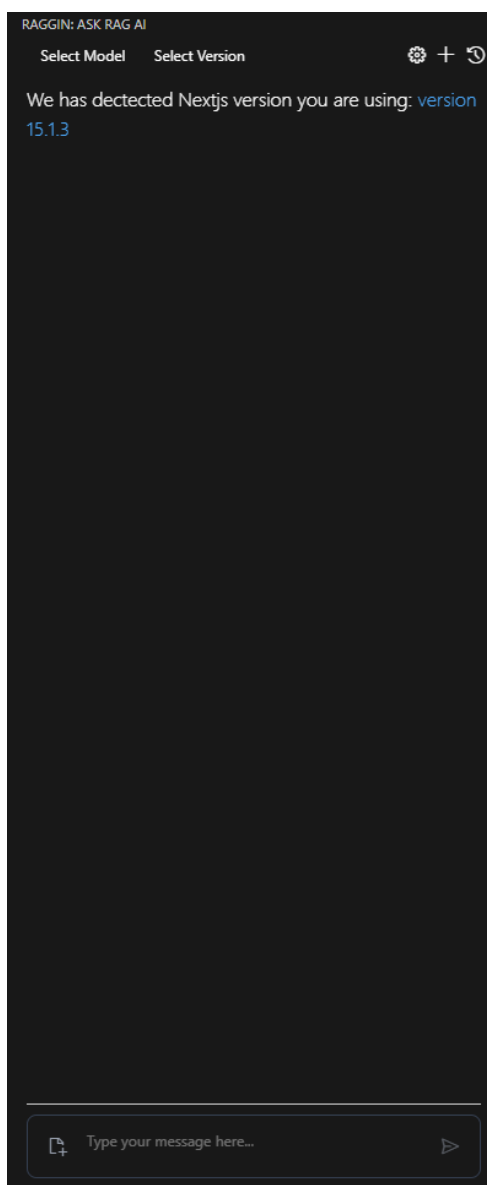


Figure A.1: Main interface when in Next.js workspace

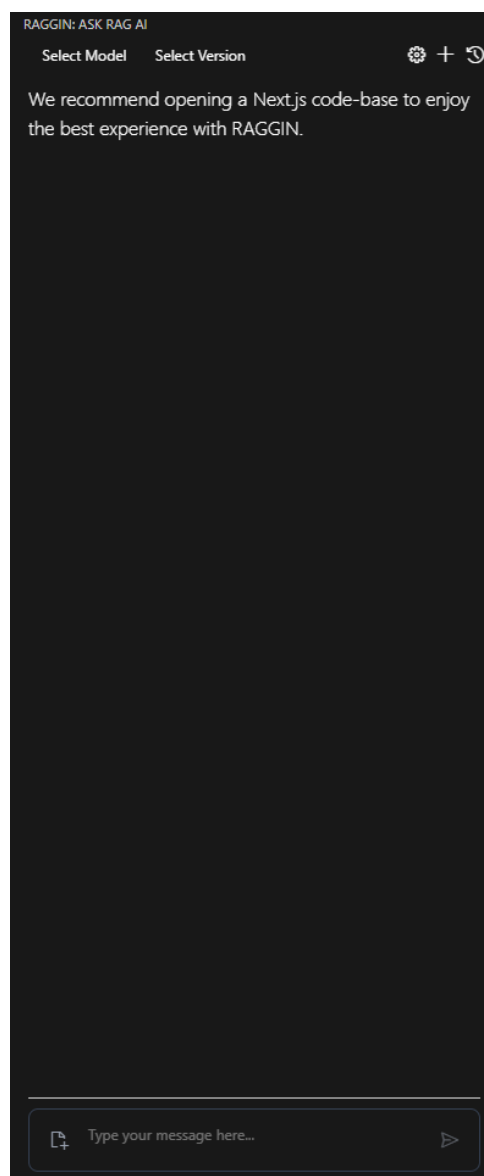


Figure A.2: Main interface when not in Next.js workspace

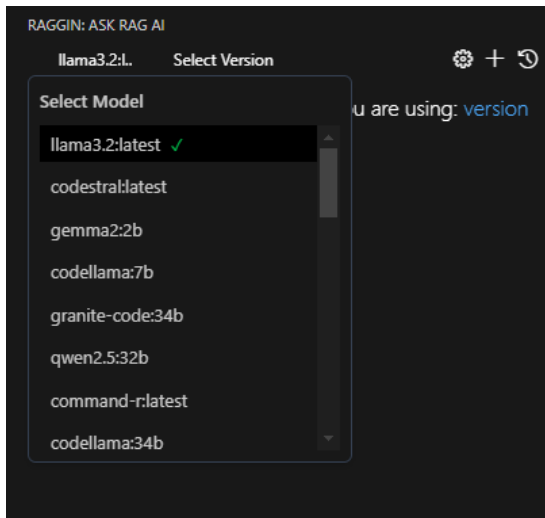


Figure A.3: Select LLM

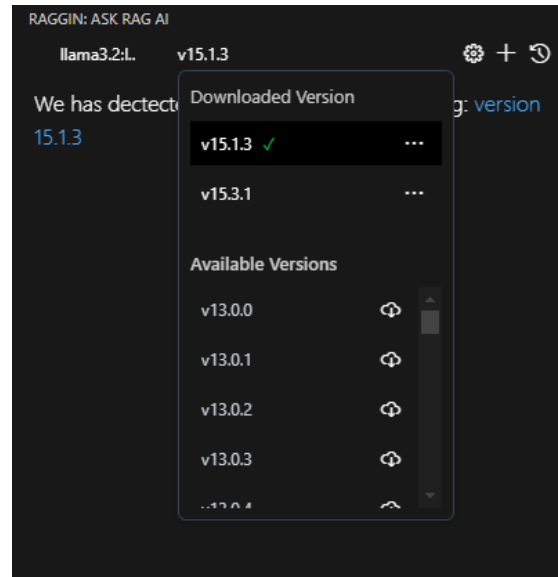


Figure A.4: Select Version

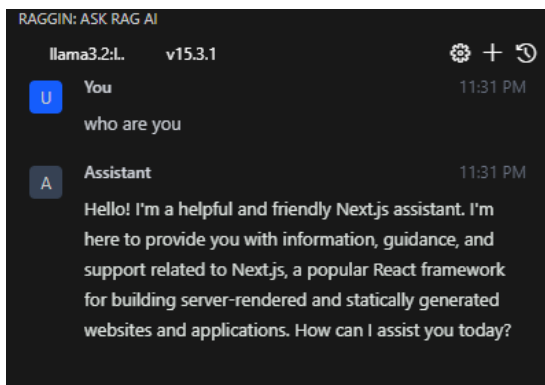


Figure A.5: RAGGIN's response to simple question

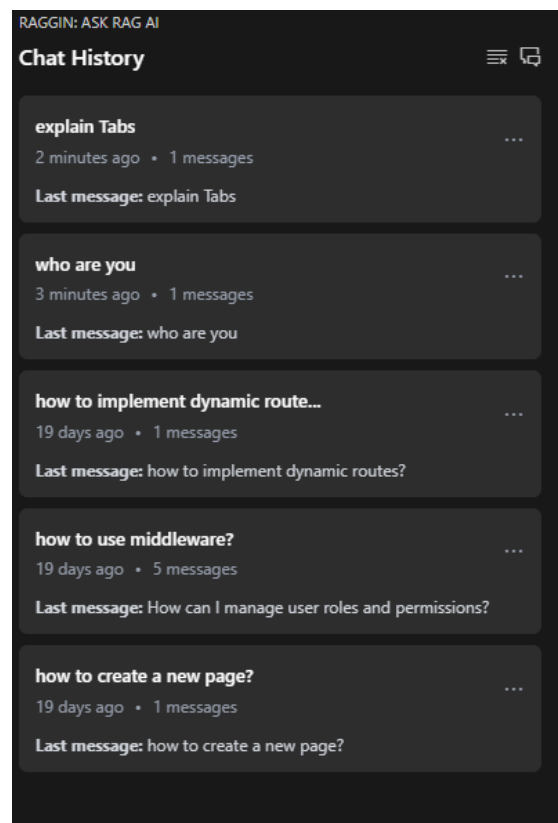


Figure A.6: Chat history

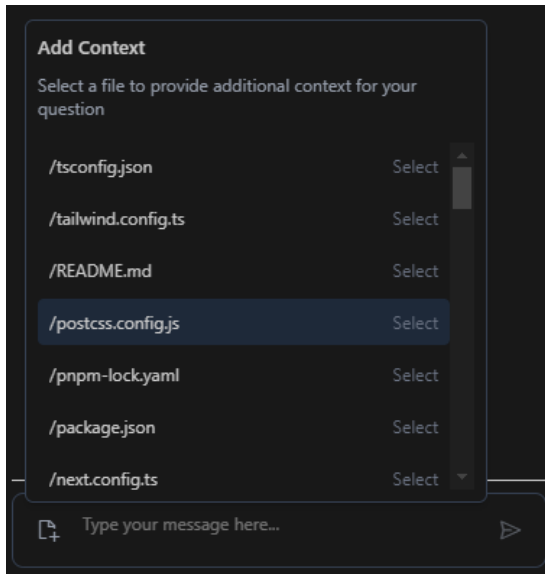


Figure A.7: Add file to context

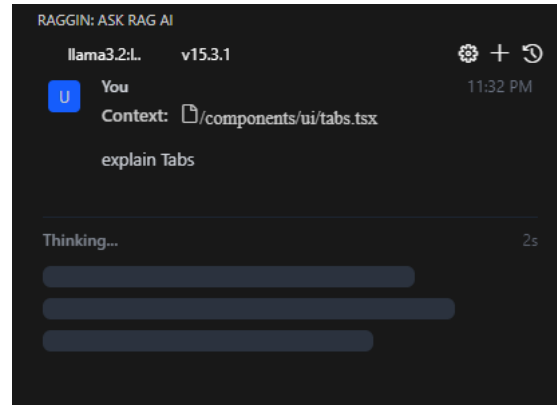


Figure A.8: Send question with selected file

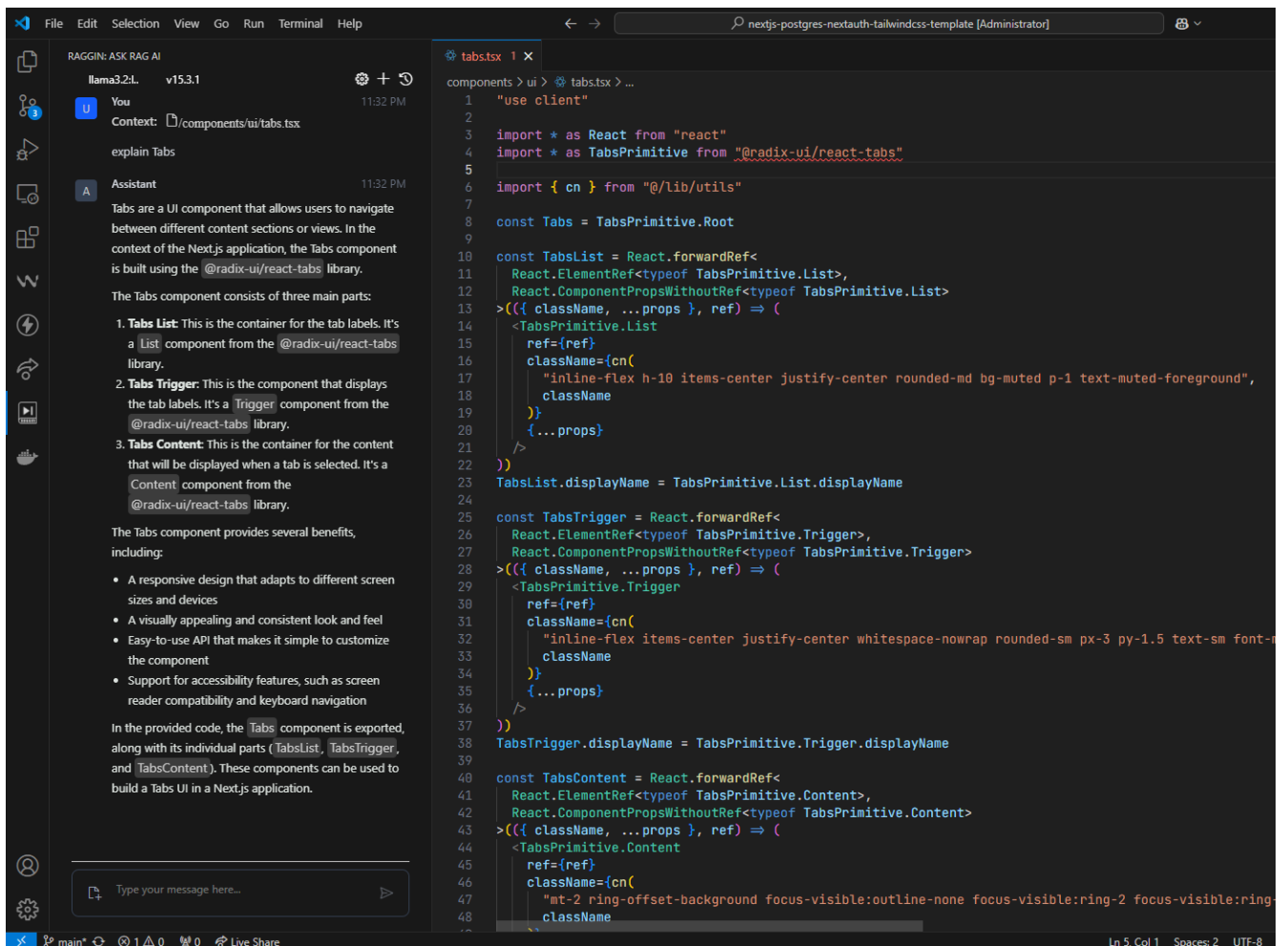


Figure A.9: RAGGIN's response to question with selected file

RAGGIN: ASK RAG AI

Advanced Settings

Retriever Options Generator Options

Weights

Sparse Weight: 0.5

Dense Text Weight: 1

Dense Code Weight: 0.9

Top K: 5

Radius and Range

Radius Sparse: 0.08 Range Sparse: 1

Radius Dense Text: 0.6 Range Dense Text: 1

Radius Dense Code: 0.6 Range Dense Code: 1

Filter

Iterative Filter

Filter Expression

Cancel Save

Figure A.10: Retriever options

RAGGIN: ASK RAG AI

Advanced Settings

Retriever Options Generator Options

Generator Settings

Microstat: 0 Microstat Eta: 0.1

Mirostat Tau: 5 Num Context: 2048

Repeat Last N: 33 Repeat Penalty: 1

Temperature: 0.8 Seed: 42

Stop Sequence: Num Predict: 1000

Top K: 100 Min P: 0.05

Top P: 0.9

Cancel Save

Figure A.11: Generator options

Chat History

Refactor code 2
1 day ago • 1 messages

Last message: Hi refactor this code "" import * as vsc
"ollama"; export asy...

talk more about this: Server-S...
3 days ago • 1 messages

Last message: talk more about this: Server-Side Rendering (SSR)

give me information about rout...
3 days ago • 5 messages

Rename chat

Fix bug

OK Cancel

Figure A.12: Rename a conversation

Evaluation and test results

| K | Precision@K | Recall@K | Retriever Latency (s) | Generator Latency (s) |
|---|-------------|----------|-----------------------|-----------------------|
| 1 | 0.27 | 0.05 | 1.18 | 0.64 |
| 3 | 0.44 | 0.36 | 1.17 | 5.74 |
| 5 | 0.57 | 0.81 | 1.21 | 5.44 |
| 7 | 0.37 | 0.74 | 1.18 | 6.00 |

Table B.1: Evaluation results with simple queries

| K | Precision@K | Recall@K | Retriever Latency (s) | Generator Latency (s) |
|---|-------------|----------|-----------------------|-----------------------|
| 1 | 0.07 | 0.02 | 3.40 | 10.82 |
| 3 | 0.24 | 0.12 | 3.48 | 11.46 |
| 5 | 0.29 | 0.24 | 3.49 | 12.13 |
| 7 | 0.21 | 0.24 | 3.49 | 11.77 |

Table B.2: Evaluation results with complex queries

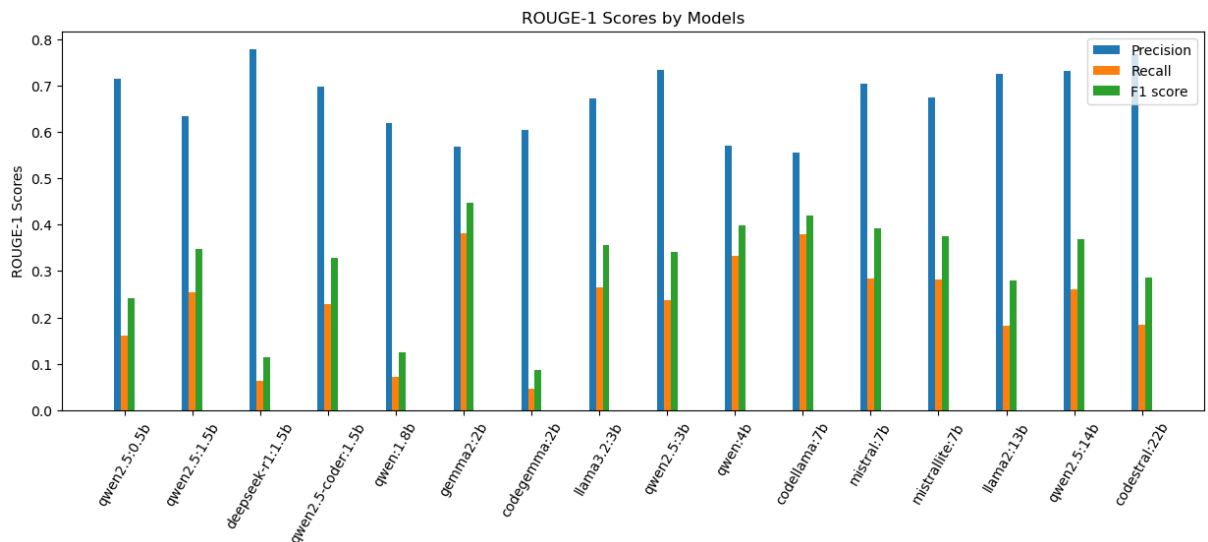


Figure B.1: ROUGE-1 Scores by LLMs

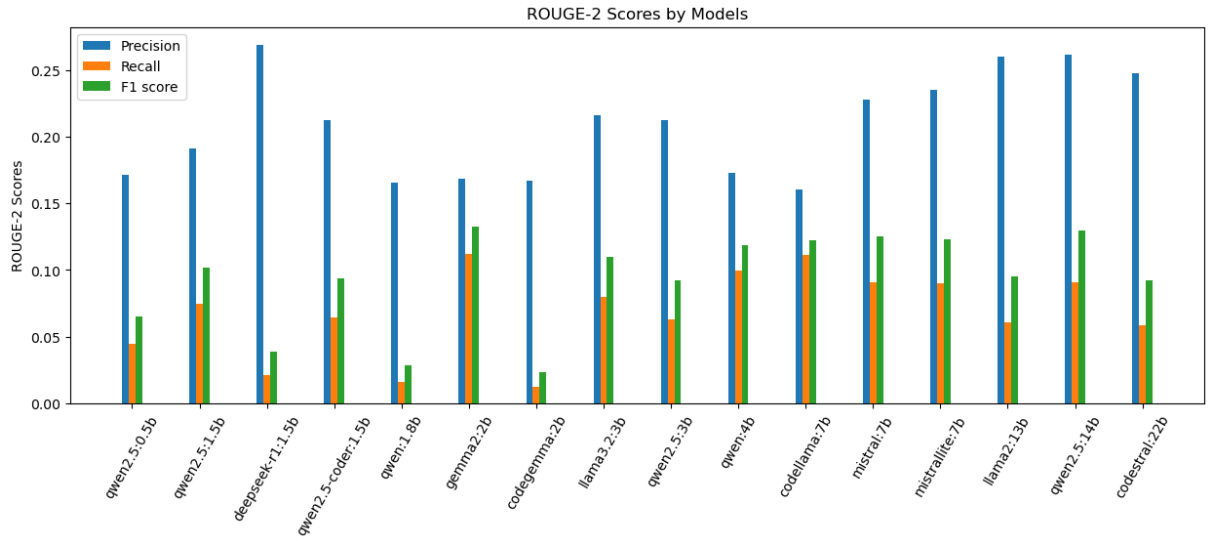


Figure B.2: ROUGE-2 Scores by LLMs

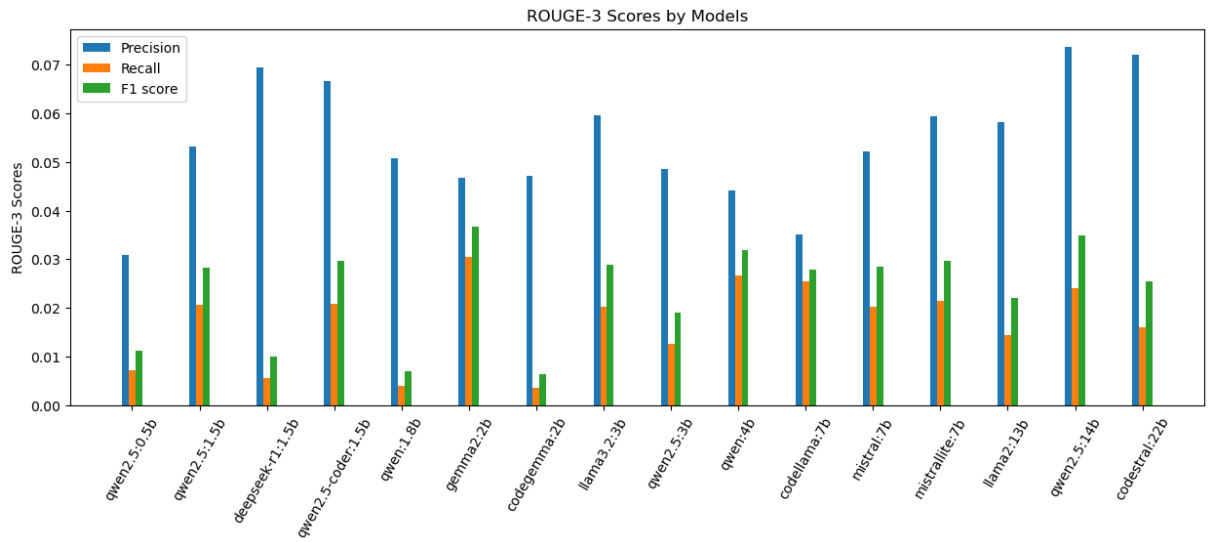


Figure B.3: ROUGE-3 Scores by LLMs

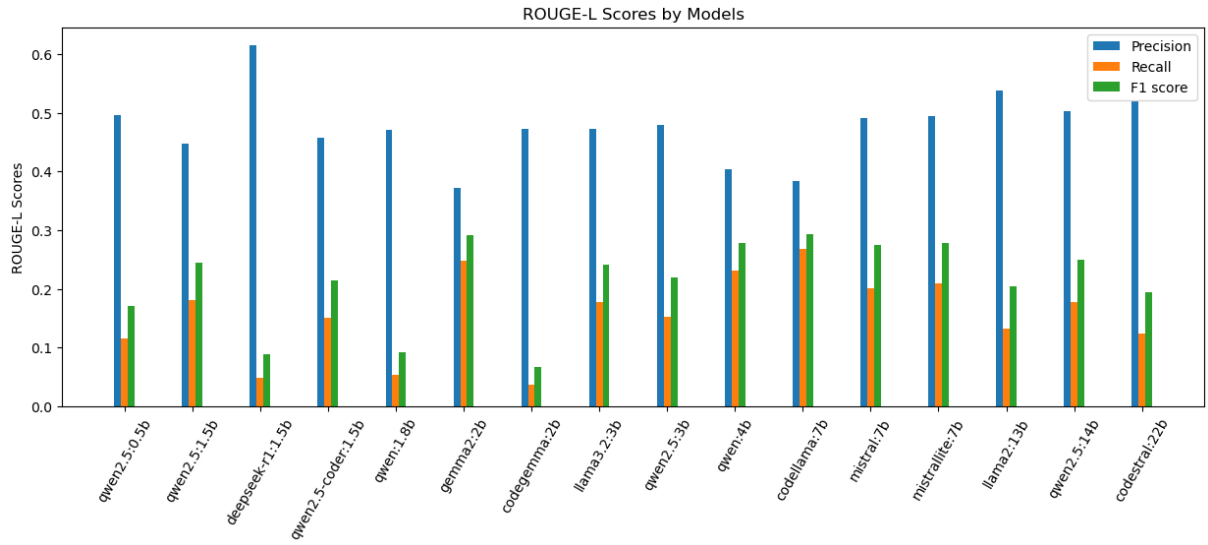


Figure B.4: ROUGE-L Scores by LLMs

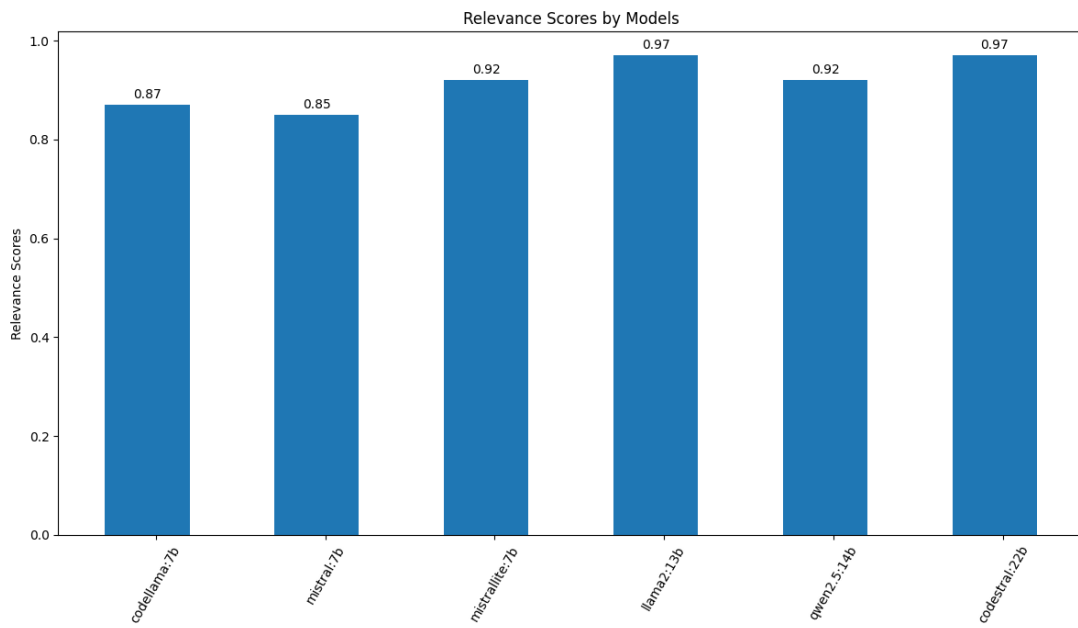


Figure B.5: Relevance Scores by some LLMs

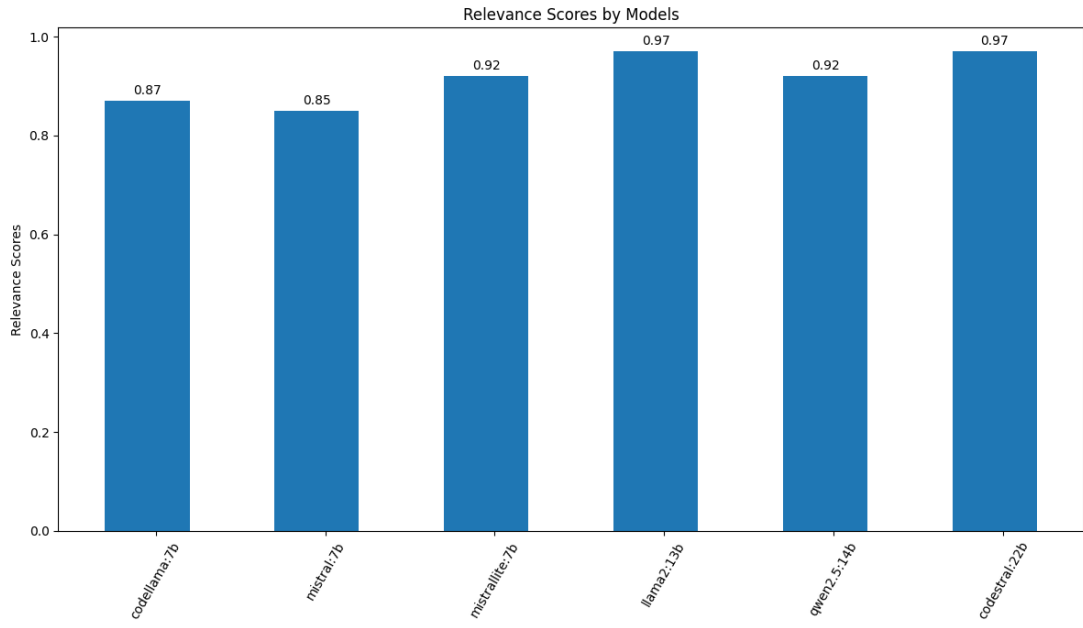


Figure B.6: Relevance Scores by larger LLMs